



Computer Science and Information Technology



Pattern Recognition (AIP312)

For Third Level Students

Prepared by

DR. DOAA EL-MORSHEDY

Lecturer of Computer Science

2026

Contents

Chapter 1: Introduction of Pattern Recognition	1
1.1 Overview of Pattern Recognition	1
1.2 Introduction to Pattern Recognition	3
What is a Pattern?	4
Why is Pattern Recognition Important?	4
1.3 Classification	5
Questions:	10
Chapter 2: Classification	12
2.1 The Classification Process	13
2.2 Image Segmentation Methods	14
2.3 Features	16
Structural Features	16
Shape Features	17
2.4 Euler Number and Statistical Moments	18
2.5 Training and Learning	21
2.6 Supervised Learning and Algorithm Selection	22
2.7 Selecting a Learning Algorithm and Evaluating Classifier Performance	23
2.8 Approaches to Classification	24
Nonmetric Approaches	24
Chapter 3 – Nonmetric Methods	28
3.1 Introduction	29
3.2 Decision Tree Classifier	29
Information, Entropy, and Impurity	32
Information Gain	36
Questions	41
Chapter 4: Supervised Learning	42
4.1 Parametric and Non-parametric Learning	43
4.2 Parametric Learning	43
Chapter 5: Supervised Learning	60
5.1 Clustering	61
5.2 Types of Clustering Algorithms	64
5.3 K-Means Clustering	43

Questions.....	58
Chapter 6:Practical Questions:	88

EPNU - 23411014
2026/05/07 03:28

Chapter 1

Introduction to Pattern Recognition

1.1 Overview of Pattern Recognition

Humans have a natural ability to recognize patterns and objects with remarkable accuracy. This skill is so automatic that we rarely think about the complex processes involved. For example, we can easily tell the difference between a human voice and a musical instrument, distinguish a handwritten '3' from an '8', or recognize the scent of a rose compared to that of an onion.

We identify faces every day without consciously analyzing the features that make them recognizable. However, although humans perform this task effortlessly, explaining the exact steps involved is difficult. This makes it challenging to design computer programs that can perform the same recognition tasks.

A human face can be viewed as a pattern made up of specific components such as the eyes, nose, and mouth arranged in particular positions. By examining many face images, a computer system can learn the structural characteristics that define a face. When the system identifies a new image as belonging to the 'face' category, it is performing pattern recognition. If it assigns the image to one category among several possible categories, the process is known as classification.

In pattern recognition, the word *pattern* is used in a broad sense. It does not necessarily mean repetition. Instead, it refers to any object or entity that needs to be identified or categorized, such as fruits, speech signals, fingerprints, or images.

A *class* is a group of objects that share similar properties, although they are not exactly the same. Each class can be distinguished from other classes based on its defining characteristics.

Pattern recognition and classification have gained significant attention due to modern applications that require advanced computational techniques. One important application area is data mining, which involves analyzing large datasets to extract useful and relevant information. Examples include fraud detection, financial prediction, and credit evaluation.

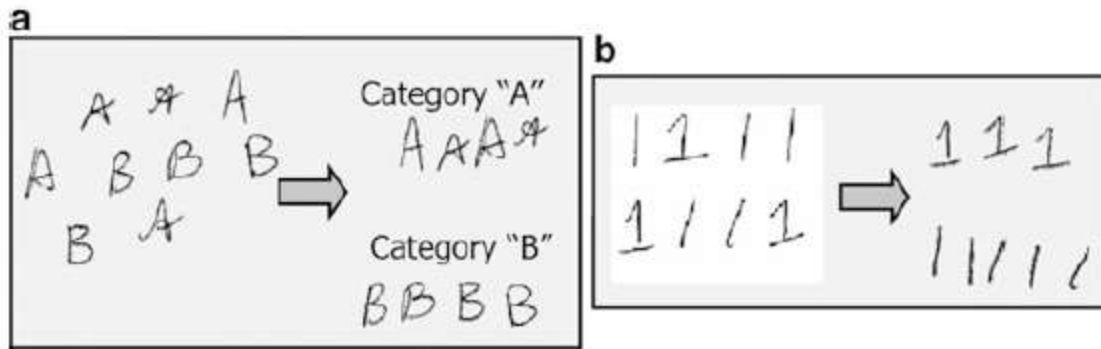


Fig. 1.1 Classification when the classes are (a) known and (b) unknown beforehand

These applications cover a wide range of important fields, including:

- ¶ **Data mining**, which involves analyzing large datasets to extract meaningful and valuable information, such as detecting fraud, predicting financial trends, and evaluating credit risk.
- ¶ **Biometrics**, where individuals are identified using unique physical characteristics such as facial features, iris patterns, or fingerprints.
- ¶ **Machine vision**, for example in automated inspection systems used on production lines.
- ¶ **Character recognition**, such as automatic postal sorting using zip codes or check-reading systems in ATMs.
- ¶ **Document recognition**, including identifying whether an email is spam by analyzing its content and header information.
- ¶ **Computer-aided diagnosis**, which assists medical professionals in making decisions by analyzing medical data like mammograms, ultrasound images, ECGs, and EEGs.
- ¶ **Medical imaging**, such as distinguishing between malignant and benign cells using MRI scans, or identifying emotional and cognitive states from functional MRI images of brain activity.
- ¶ **Speech recognition**, which can help individuals with disabilities operate machines or communicate more easily.
- ¶ **Bioinformatics**, including DNA sequence analysis to identify genes associated with specific diseases.
- ¶ **Remote sensing**, for applications such as monitoring land use and estimating crop production.

- ¶ **Astronomy**, where galaxies are classified based on their shapes, or large datasets from radio telescopes are analyzed in projects like SETI to search for possible artificial signals from space.

The techniques applied in these areas have evolved across multiple disciplines, often independently of one another. In statistics, the process of drawing general conclusions from specific observations is known as *inference*, learning from training data is referred to as *estimation*, and classification is commonly called *discriminant analysis*.

In engineering, classification is typically described as *pattern recognition*, where the methods are generally nonparametric and strongly empirical in nature. Additional approaches originate from fields such as machine learning, artificial intelligence, artificial neural networks, and data mining.

By integrating methods from these diverse disciplines, it becomes possible to present a more comprehensive and unified framework for understanding pattern recognition and classification.

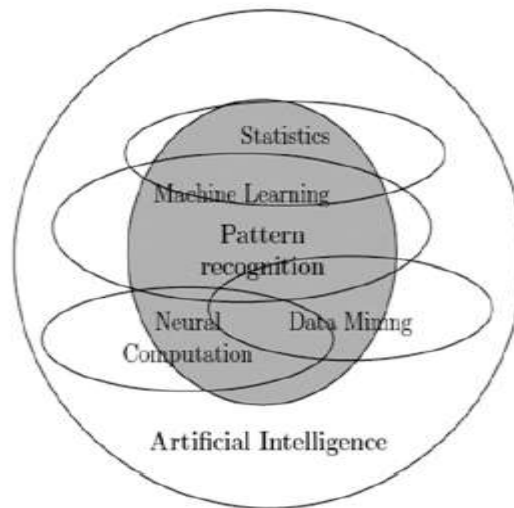


Fig. 1.2 Pattern recognition and related fields

1.2 Introduction to Pattern Recognition

Pattern Recognition is a branch of artificial intelligence and machine learning that focuses on identifying regularities, structures, or patterns in data. It enables computers to automatically recognize and classify data based on learned features.

In simple terms, pattern recognition allows a system to look at input data, detect similarities, and make decisions based on those similarities.

What is a Pattern?

A **pattern** is any object, structure, or event that can be observed, described, and analyzed in order to extract meaningful information.

In pattern recognition, the word *pattern* does not necessarily mean something that repeats regularly. Instead, it refers to **any entity that can be represented by features and classified into a category**.

A pattern can be:

- ¶ An image (such as a face or fingerprint)
- ¶ A sound signal (such as speech)
- ¶ A handwritten character (like the number 3)
- ¶ A medical signal (such as an ECG waveform)
- ¶ A DNA sequence
- ¶ Even a financial transaction record

Each pattern is described using a set of **features** (also called attributes), which are measurable properties of the object. For example:

- ¶ A face pattern may include eye distance, nose shape, and mouth position.
- ¶ A speech pattern may include frequency and amplitude.
- ¶ A handwritten digit may include stroke shape and orientation.

Patterns belonging to the same category share similarities, but they are not exactly identical. That is why pattern recognition focuses on identifying similarities and differences between patterns.

Why is Pattern Recognition Important?

Pattern recognition is important because it allows computers and intelligent systems to **analyze data, identify meaningful patterns, and make decisions automatically**.

Pattern recognition plays a central role in modern science, engineering, and everyday technology. It provides the foundation for systems that can interpret data, identify regularities, and make intelligent decisions. As the volume of digital data continues to grow rapidly, the ability to automatically analyze and classify information has become increasingly essential.

Humans are naturally skilled at recognizing patterns. We can easily distinguish between different voices, identify faces in a crowd, read handwritten text, and detect emotions from facial expressions. However, enabling computers to perform similar tasks requires systematic methods and algorithms. Pattern recognition provides these methods.

One of the primary reasons pattern recognition is important is automation. Many tasks that once required human observation and judgment can now be performed automatically. For example, machines can recognize speech commands, identify fingerprints, detect defects in manufacturing processes, and classify medical images. This automation increases efficiency, reduces human error, and saves time.

Another important aspect is decision making. Pattern recognition systems analyze data and assign it to predefined categories or classes. In banking, such systems detect fraudulent transactions. In healthcare, they assist doctors by analyzing medical scans to identify diseases. In cybersecurity, they detect suspicious activities. These systems support critical decisions in real time.

Pattern recognition is also essential for handling large volumes of data. Modern applications such as social media platforms, scientific research, financial markets, and remote sensing generate massive datasets. Without automated pattern analysis, extracting meaningful information from such data would be nearly impossible.

Furthermore, pattern recognition forms the foundation of artificial intelligence and machine learning. Many AI systems rely on recognizing patterns in data to learn from experience. Applications such as computer vision, speech recognition, natural language processing, and robotics all depend on pattern recognition techniques.

1.3 Classification

Classification is often the final stage in a complete pattern recognition process. It involves assigning objects to specific categories or classes based on their characteristics.

In image analysis, for example, an image is first segmented so that individual objects are separated from one another and from the background. Each isolated object is then labeled according to the class to which it belongs.

A typical pattern recognition system consists of several main components: a sensor to capture data, a preprocessing stage, a segmentation process, a feature extraction mechanism, training data (previously classified examples), and a classification algorithm. Each component plays a crucial role in ensuring accurate results.

The feature extraction stage is particularly important because it reduces complex raw data into measurable characteristics, such as size, shape, texture, or intensity. Instead of analyzing the entire image or dataset, the system focuses only on meaningful features. These feature values are then provided to a classifier, which evaluates them and determines the appropriate class. For instance, in medical imaging, features extracted from a tumor may help classify it as benign or malignant.

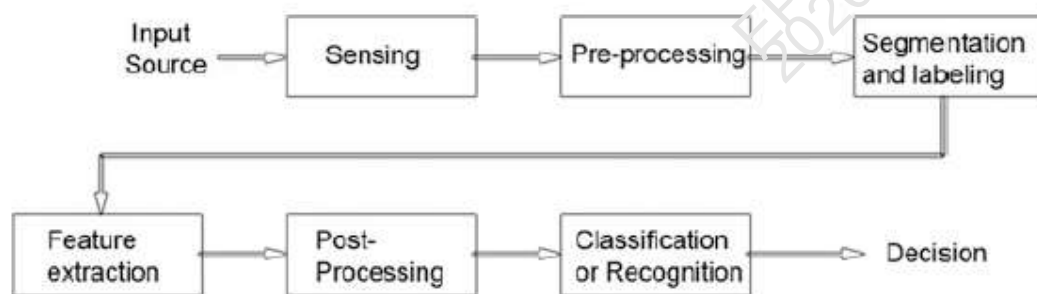


Fig. 1.3 A general classification system

The quality of classification strongly depends on the quality of the input data. In imaging systems, data quality is influenced by factors such as resolution, sensitivity, bandwidth, and signal-to-noise ratio. Therefore, preprocessing is often necessary before segmentation. Preprocessing may include image enhancement techniques such as brightness adjustment, contrast improvement, smoothing, and edge detection. Examples from the same class should have similar feature values, while examples from different classes should have different feature values, i.e., good features should have small intra-class variations and large inter-class

variations (in fig 1.4), In some cases, image restoration methods such as filtering or correction techniques are applied to improve image quality. These steps may occur sequentially, and in advanced systems, feedback mechanisms allow later processing stages to refine earlier ones through iterative loops.

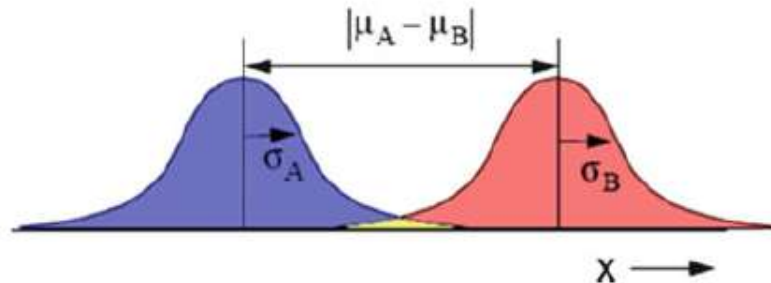


Fig. 1.4 A good feature, x , measured for two different classes (blue and red) should have small intra-class variations and large inter-class variations

An important concept in classification is the quality of features. Good features clearly distinguish between classes. Objects belonging to the same class should have similar feature values (small intra-class variation), while objects from different classes should differ significantly (large inter-class variation). In some cases, features are transformed into a new feature space to improve their discriminating power before classification.

Features may be continuous (quantitative), such as height, area, or temperature, or categorical (qualitative), such as gender, zip code, or satisfaction level. Categorical data may be nominal (unordered) or ordinal (ordered). Although it is sometimes possible to convert data from one type to another—for example, by discretizing continuous data—care must be taken because different data types support different types of analysis. In scientific applications, continuous data are often preferred because they allow more mathematical operations, such as calculating means and standard deviations.

Humans are remarkably skilled at recognizing objects, even when they appear under different lighting conditions, from different angles, at different sizes, or partially hidden. Replicating this flexibility in machine vision systems is a significant challenge. Classification systems must be robust enough to handle such variations.



Fig. 1.5 Face recognition needs to be able to handle different expressions, lighting, and occlusions

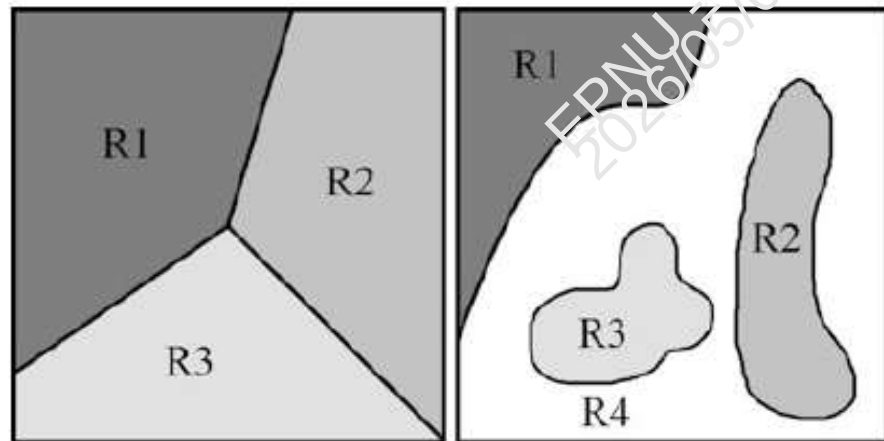


Fig. 1.6 Classes mapped as decision regions, with decision boundaries

The ultimate objective of a classifier is to assign new, unseen data (called test data) to one of the predefined classes. Each class occupies a region in what is known as the decision space, and the boundaries separating these regions are called decision boundaries. We can even recognize them when they are partially obstructed from view (Fig. 1.5). These tasks are challenging for machine vision systems in general.

The goal of the classifier is to classify new data (test data) to one of the classes, characterized by a decision region. The borders between decision regions are called decision boundaries (Fig. 1.6).

Classification techniques are generally grouped into three broad categories. The first includes **statistical methods**, which rely on quantitative features and probabilistic models. The second includes **structural or syntactic methods**, which focus on relationships and structural descriptions within patterns. The third category consists of **cognitive or hybrid methods**, such as neural networks and genetic algorithms, which combine ideas from both statistical and structural approaches. In modern applications, hybrid systems that integrate multiple classification techniques are increasingly common.

EPNU - 23411014
2026/05/07 03:28

Questions:

1. Additional Applications of Classification

List at least five real-world applications of classification that were not mentioned in the text. Consider areas such as education, transportation, cybersecurity, marketing, or environmental science.

2. Designing a Simple Classifier (Weight and Health Status)

The following data describe four adults according to their weight and health status:

Weight (kg)	Class Label
50	Unhealthy
60	Healthy
70	Healthy
80	Unhealthy

- Propose a simple rule (classifier) that correctly classifies all four adults.
- Using your rule, determine how a fifth adult weighing **76 kg** would be classified.
- Explain why your classifier makes this decision.

3. Selecting the Best Feature for Classification

The following supermarket items are described by three features: cost, volume, and color.

Item No.	Cost (\$)	Volume (cm ³)	Color	Class Label
1	20	6	Blue	Inexpensive
2	50	8	Blue	Inexpensive
3	90	10	Blue	Inexpensive
4	100	20	Red	Expensive

Item No.	Cost (\$)	Volume (cm ³)	Color	Class Label
5	160	25	Red	Expensive
6	180	30	Red	Expensive

- Which feature (cost, volume, or color) best separates the two classes?
- Justify your answer by explaining how clearly the feature distinguishes between "Inexpensive" and "Expensive" items.
- Could a combination of two features improve classification accuracy?

4. Classifying Circles and Ellipses

Suppose you are given a set of two-dimensional shapes, and you must classify each object as either a **circle** or an **ellipse**.

- What features would you extract to distinguish between circles and ellipses?
- Propose a classification rule based on these features.
- Explain how your classifier would handle imperfect or slightly distorted shapes.

EPNU - 23411014
2026/05/07 03:28

Chapter 2: Classification

2.1 The Classification Process

A typical classification system consists of several stages, arranged in a logical sequence. The process begins with data acquisition, followed by pre-processing, segmentation, and finally, classification.

The first step, **sensing or acquisition**, involves capturing the input signal using a transducer, such as a camera for images or a microphone for sound. The quality of the acquired signal is critical, as it determines how effectively features can be extracted for classification. In the case of a camera, factors such as resolution, dynamic range, sensitivity, distortion, signal-to-noise ratio, and focus all influence the quality of the captured image.

Once the data is acquired, it often undergoes **pre-processing** to improve its quality and prepare it for segmentation. Pre-processing techniques aim to reduce noise, correct uneven illumination, and handle any inconsistencies in the data. For instance, smoothing an image using a Gaussian filter can reduce the impact of random noise, while median filtering can remove salt-and-pepper noise. Adjustments such as background correction and histogram equalization are commonly applied to ensure uniform brightness and contrast. Additionally, missing or anomalous data may need to be addressed to avoid errors in subsequent stages.

The next stage, **segmentation**, divides the image into regions that are meaningful for the task at hand. This involves separating the foreground objects of interest from the background. Segmentation methods are generally divided into two categories. **Region-based approaches** group pixels that share similar characteristics, such as brightness, color, or texture. Care must be taken: overly strict similarity criteria can lead to over-segmentation, while too lenient criteria may merge distinct objects into a single region. On the other hand, **boundary-based approaches** focus on detecting edges in the image, linking discontinuities to form continuous contours around objects.

The ultimate goal of segmentation is to produce regions that correspond to objects as perceived by humans. Achieving this remains challenging, as natural images often contain noise, varying illumination, and complex textures. Consequently, designing effective segmentation and classification systems requires careful consideration of both the input data and the techniques used.

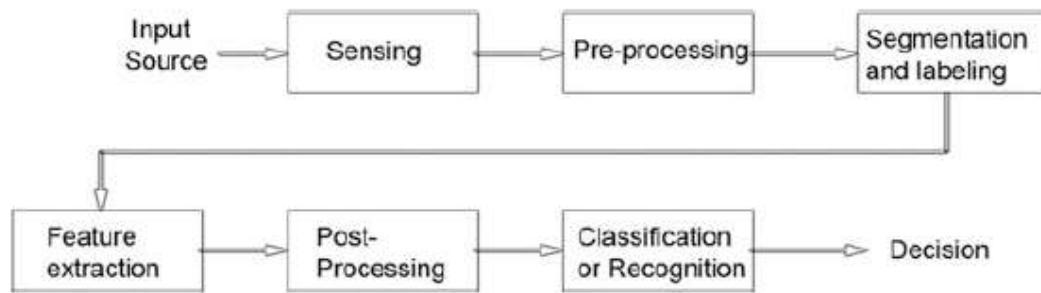


Fig. 2.1 A general classification process

2.2 Image Segmentation Methods

Image segmentation is a crucial step in pattern recognition, aiming to separate objects of interest from the background. Segmentation methods can be broadly categorized into **region-based** and **boundary-based** approaches.

Region-Based

Methods

Region-based techniques work by grouping pixels into regions that share similar characteristics. One common approach is **thresholding**, which can be applied globally or locally using adaptive methods. Optimal thresholding techniques, such as **Otsu's method**, **iso-data**, or **maximum entropy thresholding**, can help select the best threshold automatically. In cases where objects overlap after thresholding, additional techniques such as thresholding the **distance transform** or applying the **waters**

Segmentation simplifies an image by converting it into a **binary form**, where objects of interest (foreground) are separated from the background, while preserving their shape and size for further analysis. In such a binary image, foreground pixels are typically assigned a value of **1** (white), and background pixels are assigned **0** (black).

To facilitate analysis, it is often useful to **label** each object in the image with a unique number. **Connected components labeling** scans the binary image and groups pixels into components based on connectivity, meaning that all pixels within a component are connected and share similar values. Once the components are identified, each pixel is assigned a label (1, 2, 3, ...), which can then be displayed as different gray levels or colors. All pixels in a connected component share similar pixel values and are in some way connected with each other. Once all groups have been determined, each pixel is

labeled with a number (1, 2, 3, . . .), according to the component to which it was assigned, and these numbers can be looked up as gray levels or colors for display (Fig. 2.2).

Labeling enables easy counting of objects within an image. Additionally, labeled binary objects can be used to **mask the original image**, isolating each object while retaining its original pixel values. This allows measurement of individual object properties or features. Masking can be applied in several ways, such as using the binary mask as an overlay or alpha channel to hide background pixels, or by directly modifying the stored image—either by multiplying the grayscale image by the mask or performing a bitwise AND operation. This process of isolating features forms the foundation of **region-of-interest (RoI) processing**.

After segmentation, **post-processing** can further refine the image for feature extraction. For example, partial objects can be removed from around the periphery of the image (e.g., Fig. 2.2e), Partial objects near the image edges can be removed, disconnected objects merged, objects outside a specific size range eliminated, and holes in objects or background filled using morphological operations like **opening** or **closing**. These steps ensure the segmented image is clean and ready for accurate measurement and analysis.

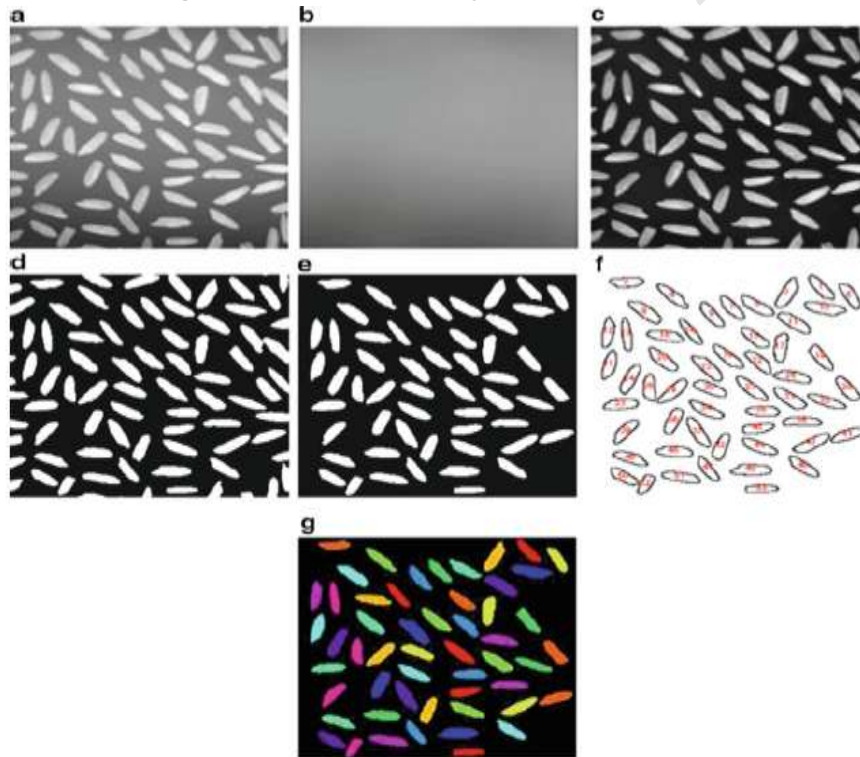


Fig. 2.2 (a) Original image, (b) variable background [from blurring (a)], (c) improved image [$=(a) - (b)$], (d) segmented image [Otsu thresholding of (c)], (e) partial objects removed from (d), (f) labeled components image, (g) *color-coded* labeled components image

2.3 Features

The next stage in the classification process is **feature extraction**. Features are distinct properties of objects that help differentiate one class from another. Ideally, the values of a feature should be similar for all objects within the same class and distinctly different from objects in other classes or from the background.

Features can take two main forms: **continuous** or **categorical**.

- ¶ **Continuous features** have numerical values, such as length, area, or texture measurements.
- ¶ **Categorical features** are labeled and can be further classified as:
 - **Ordinal:** where the order of the labels is meaningful (e.g., class ranking, military rank, or level of satisfaction).
 - **Nominal:** where the order of labels is not meaningful (e.g., names, zip codes, or department labels).

The selection of features depends on both the type of image and the application. Regardless of the context, features should possess several important qualities:

1. **Robustness:** Features should remain largely unchanged under translation, rotation, scaling, and variations in illumination. Well-designed features are also partially resistant to noise and image artifacts, though pre-processing may be required.
2. **Discriminative power:** Feature values should clearly separate different classes, ideally with little or no overlap.
3. **Reliability:** Objects of the same class should consistently exhibit similar feature values.
4. **Independence:** Features should be uncorrelated. For example, length and area are often correlated, so using both as separate features may be redundant.

Features provide higher-level representations of an object's **structure** and **shape**.

Structural Features

Structural features describe the internal composition of an object. Examples include:

- ¶ **Gray-level histogram measurements** obtained from a region of interest, such as the mean pixel value, standard deviation, contrast, or entropy.
- ¶ **Texture**, which can be characterized using statistical moments of the gray-level histogram or through its fractal dimension.

Shape Features

Shape features capture the geometry of an object and include:

- ¶ **Size or area (A):** Determined from the number of pixels comprising the object.
- ¶ **Perimeter (P):** Measured, for example, using the chain code of the object's boundary.
- ¶ **Circularity:** Defined as a ratio between the area and perimeter, commonly expressed as $4\pi A/P^2$ or similar variants.
- ¶ **Aspect ratio:** The ratio of the object's Feret diameters, which can be determined by enclosing the object in a bounding box.

Skeleton or medial axis: Provides a simplified representation of the object's shape, including branch points and end points, which can be counted based on neighboring pixels along the skeleton which can be obtained by counting the number of neighboring pixels on the skeleton (viz., 3 and 1, respectively) (e.g., Fig. 2.3).

These structural and shape features collectively allow classification systems to capture meaningful distinctions between objects in a way that mirrors human visual perception.

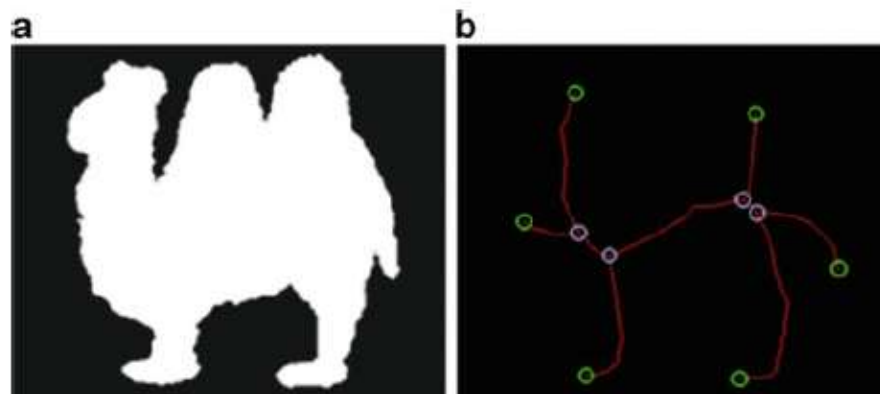


Fig. 2.3 (a) Image and (b) its skeleton (*red*), with its branch points (*white*) and end points (*green*) circled

2.4 Euler Number and Statistical Moments

Another important feature used in image analysis is the **Euler number**. This is defined as the difference between the number of connected components (i.e., individual objects) and the number of holes within the image. It provides a simple yet powerful measure of an object's topology.

In addition to topological features, **statistical moments** are widely used to describe an object's shape and spatial distribution. For a two-dimensional discrete function $f(x,y)$, such as a digital image with $M \times N$ pixels, the (m,n) -th moment is defined as:

$$m_{mn} = \sum_{x=1}^M \sum_{y=1}^N x^m y^n f(x,y)$$

where m_{00} is the sum of the pixels of an image: for a binary image, it is equal to its area. The *centroid*, or *center of gravity*, of the image, (μ_x, μ_y) , is given by $(m_{10}/m_{00}, m_{01}/m_{00})$. The central moments (viz., taken about the mean) are given by

$$\mu_{mn} = \sum_{x=1}^M \sum_{y=1}^N (x - \mu_x)^m (y - \mu_y)^n f(x,y)$$

where μ_{20} and μ_{02} are the variances of x and y , respectively, and μ_{02} is the covariance between x and y . The *covariance matrix*, C or $\text{cov}(x, y)$, is

$$C = \begin{bmatrix} \mu_{20} & \mu_{11} \\ \mu_{11} & \mu_{02} \end{bmatrix}$$

Shape features serve as the basis for computing higher-level descriptors of an object. These descriptors can capture important aspects such as orientation, elongation, and structural geometry, which are essential for distinguishing between objects with similar appearances.

For instance, in the case of nuts and bolts, some nuts may be viewed face-on while others appear edge-on. Selecting the appropriate combination of shape and structural features can effectively separate these objects from one another, as illustrated in Figure 2.4.

All the measured characteristics of an object are then combined into a **feature vector**, denoted by x , which takes the form:

$$x_1, x_2, \dots, x_n$$

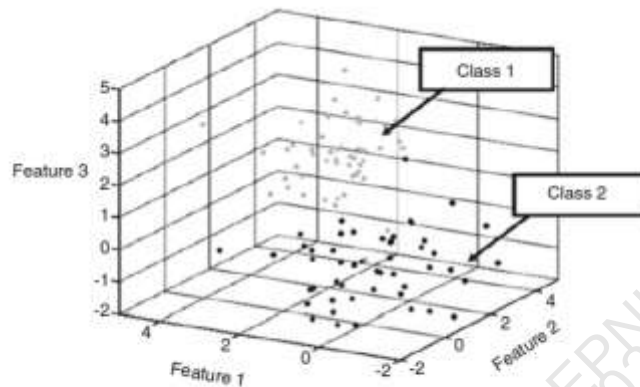


Fig. 2.5 Three-dimensional feature space containing two classes of features, class 1 (in gray) and class 2 (in black)

Each component x represents a specific feature, and together the feature vector serves as the input for classification algorithms.

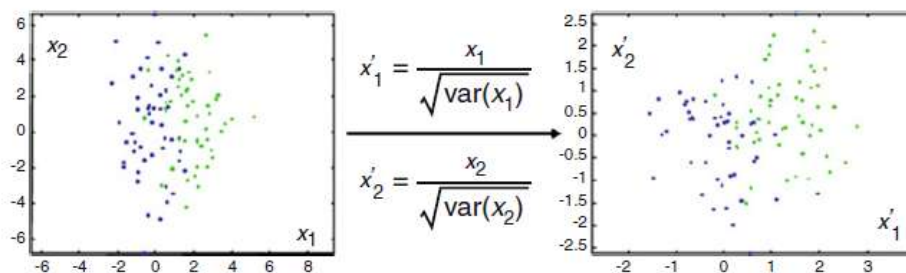


Fig. 2.6 Scaling of features

$$\begin{matrix} x = x_1 \\ x_2 \\ \vdots \\ x_n \end{matrix} \quad (2.4)$$

Feature vectors can be visualized as points in a **feature space** (see Figure 2.5). For a set of n features, the feature space is n -dimensional, with each feature representing one axis. In an ideal scenario, objects belonging to the same class form tight clusters in this space, reflecting **reliability**, while objects from different classes are well separated, demonstrating **discriminative power**.

In classification, the primary objective is to assign each feature vector to one of the predefined classes $\{\omega_i\}$. When the features have differing scales or units, it is advisable to normalize them—typically by dividing each feature by its standard deviation—so that no single feature dominates the classification process (see Figure 2.6).

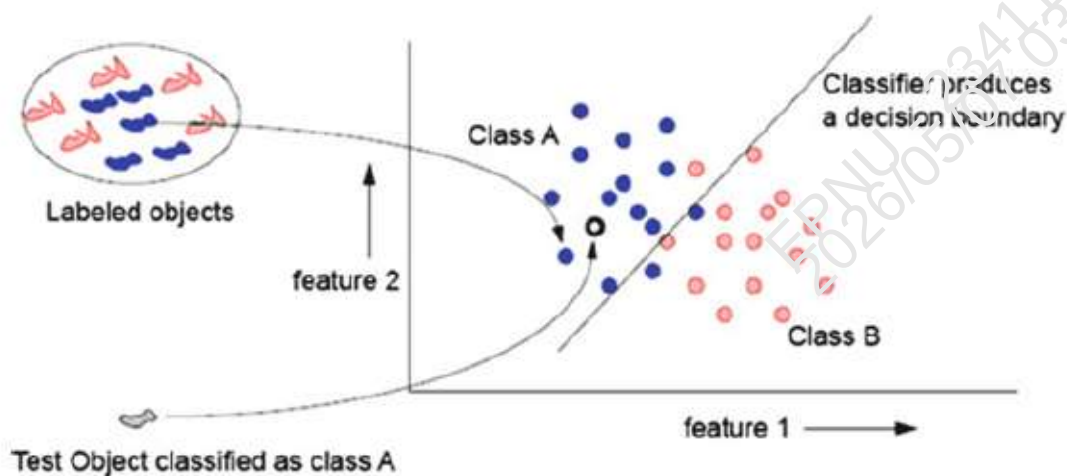


Fig. 2.7 Linear classification, using labeled training sets and two features, results in a linear decision boundary

The **classification stage** assigns objects to specific categories, or classes, based on the extracted feature information. A key question arises: how many features should be measured, and which ones are the most effective?

Measuring more features increases the dimensionality of the feature space, which complicates the classification process. Higher dimensionality demands more computation time, storage, and can make separating classes more difficult—a phenomenon known as the **curse of dimensionality**. To address this, it is often preferable to identify a minimal set of high-quality features that are sufficient for accurate classification. This approach follows the heuristic principle of **O c c a m' s r a**, which favors the simplest effective solution, or in modern terms, **KISS (Keep It Simple,**

Stupid). While simplicity may not always guarantee the best result, it provides a practical guiding bias.

A prudent strategy is to initially measure more features than may be strictly necessary and then reduce the feature set through one of two approaches:

1. **Feature Selection:** Identifying the most informative subset of features and removing irrelevant or redundant ones (Yu and Liu, 2004).
2. **Feature Extraction:** Transforming the original features into a smaller set of new, more informative features (Markovitch and Rosenstein, 2002). One of the most widely used feature extraction techniques is **Principal Component Analysis (PCA)**.

One common paradigm for classification is **learning from examples**. In this approach, a sample of labeled objects, known as the **training set**, is randomly selected, and the feature vectors of these objects are plotted in feature space. Based on this, it may be possible to construct a **classifier** capable of separating the classes using a **decision boundary** or **decision surface**.

For example, a **linear classifier** produces a decision surface that is a hyperplane (Figure 2.7). The decision boundary should be as simple as possible while still achieving effective classification. Using a labeled training set, where the class membership of each object is known, is an example of **supervised learning**.

2.5 Training and Learning

A typical classification problem involves the following task: given a set of example objects, known as the **training set**, which represent several classes, the goal is to classify new objects, called the **test set**, into one of these predefined classes. To achieve this, features must be carefully selected so that **within-class variations** are smaller than **between-class variations**.

If the class membership of the objects is known in advance, the process is referred to as **supervised learning**. If the classes are unknown and must be inferred from the data, the process is known as **unsupervised learning**. Unsupervised learning aims to discover meaningful, previously unknown classes.

The process of using data to determine the most effective set of features for a classifier is called **training the classifier**. The most effective training methods rely on learning from examples. To evaluate the usefulness of a feature set, a **performance metric** based on classification errors is calculated.

Learning, also referred to as **machine learning** or **artificial intelligence**, involves adapting the classification algorithm to improve its performance. This typically requires iterative feedback, adjusting earlier steps in the process until a desired level of accuracy is achieved. Ideally, performance improves monotonically over successive iterations (Figure 2.8), though achieving this in practice can be challenging.

A specialized form of learning, called **reinforcement learning**, focuses on discovering an optimal sequence of actions to maximize a reward. The learning system must explore and identify the best strategy to achieve the highest possible outcome.

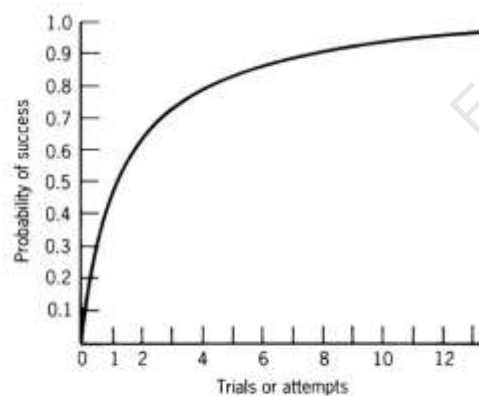


Fig. 2.8 Idealized learning curve

2.6 Supervised Learning and Algorithm Selection

Supervised learning is an inductive reasoning process in which a set of rules is inferred from example instances provided in a **training set**. Based on these rules, a **classifier algorithm** is either chosen or designed to correctly apply the learned rules to new, unseen instances.

The overall process of applying supervised learning to solve a practical problem is illustrated in Figure 2.9. This framework guides the selection of appropriate algorithms and ensures that the classifier generalizes effectively from the training data to the test data.

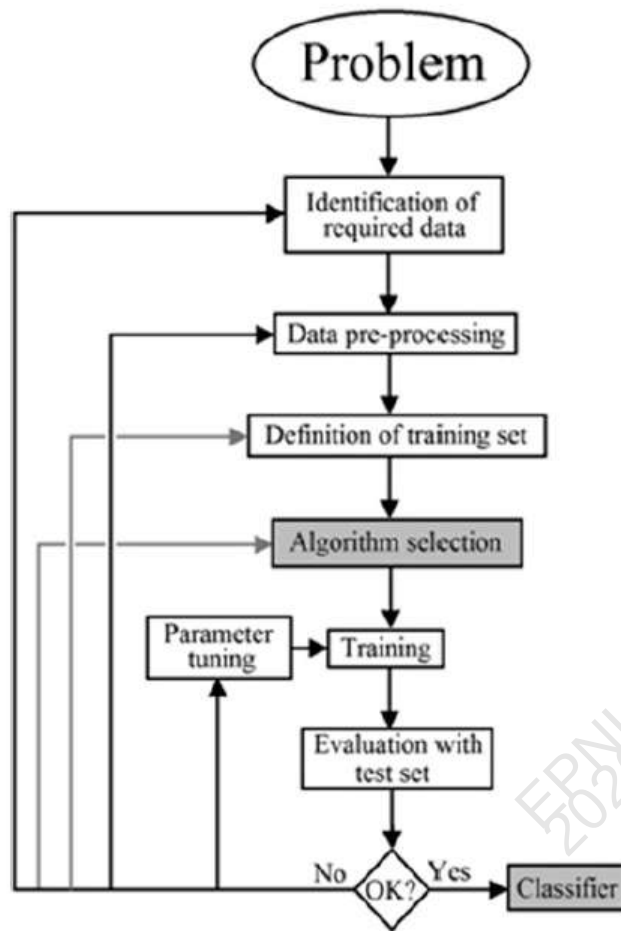


Fig. 2.9 The process of supervised learning

2.7 Selecting a Learning Algorithm and Evaluating Classifier Performance

Choosing the appropriate learning algorithm is a critical step in supervised learning. Once an algorithm is selected, it should be applied to the **training set**, and its performance must be evaluated before it is adopted for general use.

The most common measure of performance is **prediction accuracy**, defined as the percentage of correct predictions relative to the total number of predictions. There are several techniques for estimating the accuracy of a classifier:

1. **Simple Split:** Divide the training set into two portions, typically using two-thirds of the data for training and the remaining one-third for evaluating performance.
2. **Cross-Validation:** Partition the training set into mutually exclusive, equal-sized subsets. For each subset, train the classifier on the union of all the other subsets,

then test on the excluded subset. The average error rate across all subsets provides an estimate of the classifier's overall error rate.

3. **Leave-One-Out Validation:** A special case of cross-validation where each test subset contains only a single instance. While computationally intensive, this method provides the most precise estimate of a classifier's error rate, making it useful when accuracy is paramount.

By carefully selecting the algorithm and rigorously evaluating its performance, one can ensure that the classifier generalizes well to unseen data and achieves reliable results in practical applications.

2.8 Approaches to Classification

Classification problems can be tackled using a variety of approaches, depending on the nature of the data, the desired level of accuracy, and the computational resources available. The choice of approach influences both the design of the classifier and its performance.

1. Statistical Classification:

Statistical methods assume that the features of each class follow a specific probability distribution, often Gaussian. The classifier then assigns a new object to the class that maximizes the likelihood of observing its feature values. These methods are mathematically rigorous and often provide insight into the underlying data structure.

Nonmetric Approaches

Nonmetric approaches to classification include **decision trees**, **syntactic (or grammatical) methods**, and **rule-based classifiers**. These methods are particularly well-suited for **categorical data**, where features are expressed as discrete attributes rather than numerical values.

A natural and intuitive way to classify a pattern is by asking a **series of questions**, where each subsequent question depends on the answer to the previous one. This approach works especially well for nonmetric data because the questions can be framed to require

simple **yes/no** or **true/false** responses. However, it can also be applied to quantitative data if the questions involve numerical thresholds.

The sequence of questions can be visualized as a **decision tree** (Figure 2.11). In this tree structure:

- ¶ **Decision Nodes** represent questions about a particular feature.
- ¶ **Branches** correspond to the possible answers or outcomes of each question.
- ¶ **Leaf Nodes** are the terminal nodes, which indicate the final **classification** of the pattern.

This hierarchical structure allows for a step-by-step classification process, making the reasoning behind the classification transparent and easy to interpret.

For example, when classifying fruits, the first node might ask, "Is the color red?" If yes, the next node could ask, "Is the size large?" The process continues until the fruit is classified as an apple, cherry, or another category at a leaf node.

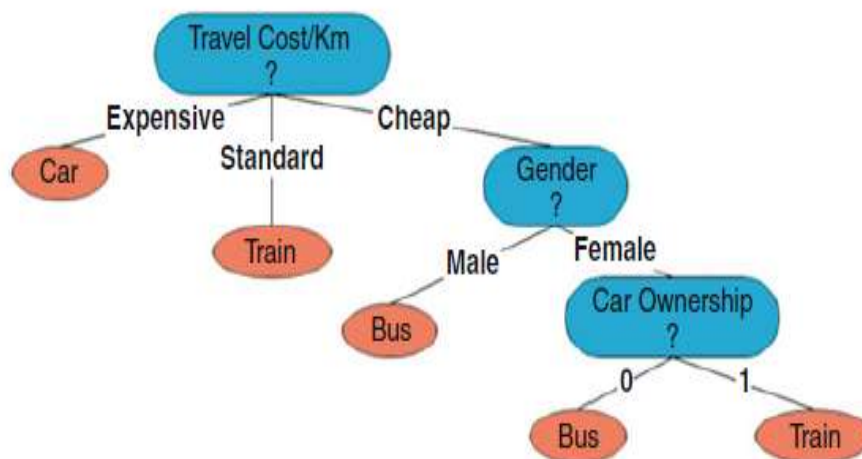


Fig. 2.11 A decision tree (the decision nodes are colored blue, and the leaf nodes orange)

2. Nearest Neighbor Methods:

3. These methods classify an object based on the class of its nearest neighbors in the feature space. The simplest form is the **k-nearest neighbor (k-NN)** algorithm, where the majority class among the k closest points determines the classification.

These approaches are non-parametric and make few assumptions about the data distribution.

4. Neural Networks:

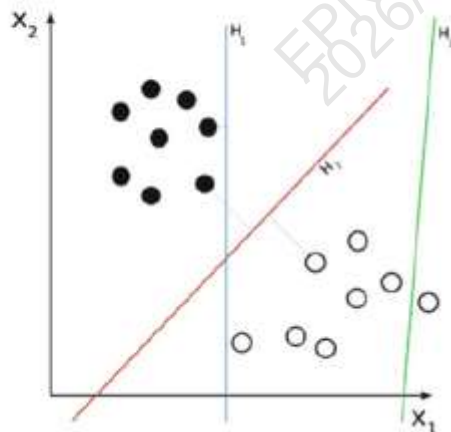
Neural networks, inspired by the structure of the human brain, are capable of learning complex nonlinear relationships between features and classes. Through iterative training and weight adjustment, they can model highly intricate decision boundaries.

5. Support Vector Machines (SVM):

SVMs aim to find the optimal hyperplane that maximizes the margin between classes in feature space. They are particularly effective for high-dimensional data and cases where classes are not linearly separable.

2.6 Examples

Fig. 2.12 A linear SVM in 2D feature space. H_1 separates the two classes with a small margin, but H_2 separates them with the maximum margin (H_3 doesn't separate the two classes at all)



6. Ensemble Methods:

Ensemble methods combine multiple classifiers to improve performance. Techniques such as **bagging**, **boosting**, and **random forests** aggregate the predictions of individual models to achieve greater accuracy and robustness.

Each classification approach has its strengths and limitations, and the choice often depends on the specific problem, the size and quality of the dataset, and the computational constraints. In practice, it is common to experiment with multiple approaches and evaluate their performance before selecting the most suitable method.

Questions:

1. What is a **feature** in image classification? Give an example.
2. What is the difference between **continuous** and **categorical** features?
3. What is a **feature vector**?
4. What does **feature space** mean?
5. What is the **Euler number** in an image?
6. Give an example of a **structural feature**.
7. Give an example of a **shape feature**.
8. What is the main goal of **classification**?
9. What is the difference between **supervised** and **unsupervised learning**?
10. What is a **training set**, and why is it important?
11. Name one method to evaluate the **accuracy of a classifier**.
12. What is meant by the "curse of dimensionality"?
13. Why is **feature selection** important in classification?
14. What is **feature extraction**, and how is it different from feature selection?
15. Give an example of a **real-world application** of image classification

Chapter 3 – Nonmetric Methods

3.1 Introduction

Nonmetric methods are used to handle **categorical data**, where the features of objects are described as attributes rather than numerical values. For instance, a fruit might be represented by a set of attributes such as:

{color = red, texture = shiny, taste = sweet, size = large}

Similarly, a segment of DNA could be represented as a sequence of base pairs, for example:

ñGACTTAGATTCCAò

These types of discrete data are not suitable for methods that rely on numerical calculations, such as distance-based classifiers. Instead, they are conveniently handled by **decision trees**, **rule-based classifiers**, and **syntactic (grammar-based) methods**, which can directly process categorical attributes.

3.2 Decision Tree Classifier

A **decision tree classifier** is a hierarchical model that partitions the feature space based on categorical attributes. At each node of the tree, a decision is made according to the value of a specific feature, guiding the path to the next node or leaf. The leaves of the tree correspond to the predicted classes.

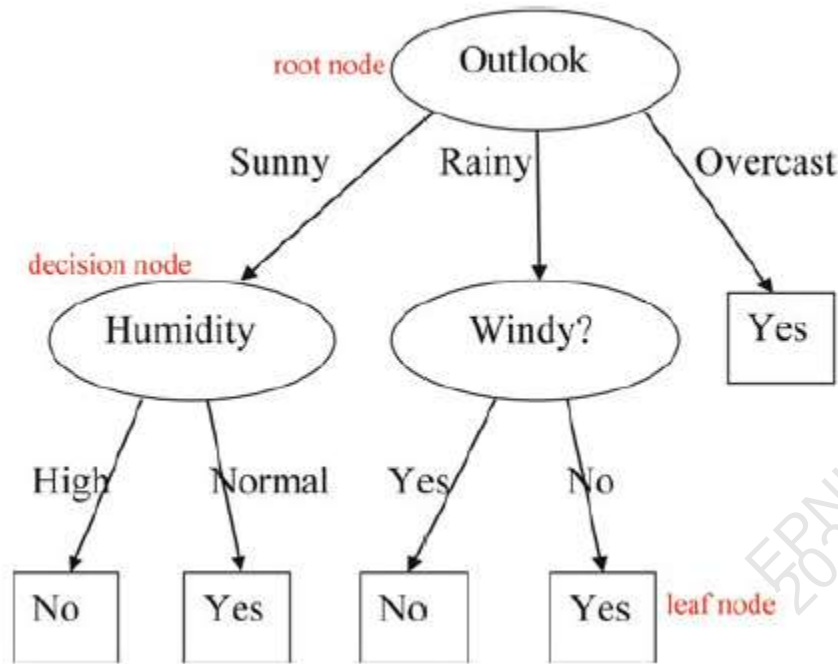
Decision trees are particularly effective for nonmetric data because they can handle **discrete attributes** directly and provide an intuitive, interpretable structure for classification rules.

A **decision tree classifier** is a hierarchical model used to classify objects based on their features. It is especially well-suited for **categorical (nonmetric) data**, where features are described by discrete attributes rather than numerical values.

The tree consists of three main components:

1. **Root Node:** This is the top node of the tree, representing the entire dataset. The root node splits the data based on the most informative feature.

2. **Internal Nodes:** Each internal node represents a test on a feature, with branches corresponding to possible outcomes (e.g., ñredò vs. ñgreenò for color).
3. **Leaf Nodes:** These are the terminal nodes of the tree, representing the predicted class of objects that reach that node.



How Decision Trees Work

The classification process starts at the **root node**. The value of a specific feature determines which branch the object follows. This process continues recursively through the **internal nodes** until a **leaf node** is reached, which assigns the object to a class.

For example, consider a dataset of fruits with features such as color, texture, and size. A decision tree might first split the data based on color, then further divide each color group based on size, and finally classify the fruits at the leaves as apples, oranges, or cherries.

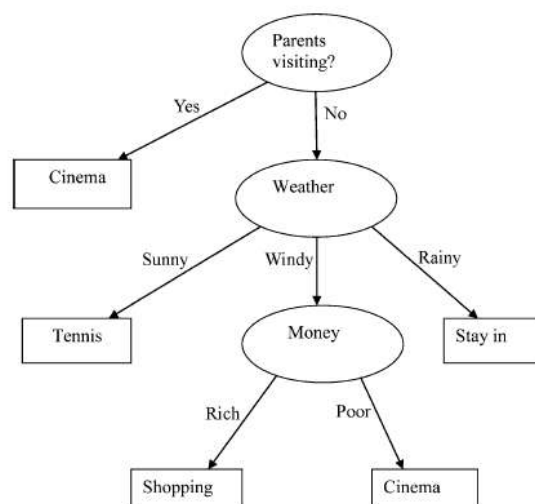
a **three-level decision tree** that models a simple decision-making scenario: deciding what to do on a Saturday morning. For instance, if the sun is shining and our parents have not arrived, the decision tree recommends going out to play tennis.

One key feature of decision trees is that they **cover all possible eventualities**. In this example, every combination of weather conditions, parental presence, and financial situation is accounted for, ensuring that no scenario is left undefined.

Decision trees are not limited to representing everyday decisions – they are a **general tool for classification problems**. By rephrasing the questions in terms of object features, the same tree structure can be applied to assign patterns or objects to specific classes.

Some advantages of decision tree classifiers include:

- ¶ **Compatibility with nonmetric/categorical data:** Decision trees handle discrete attributes naturally, including nominal features with no inherent ordering. They can also be adapted for quantitative data.
- ¶ **Clear interpretability:** The hierarchical structure provides an intuitive explanation for the classification, and the decision rules can be easily converted into logical expressions.
- ¶ **Efficient computation:** Once the tree is constructed, classifying new instances is very fast, as it involves only a series of simple tests along the branches of the tree.
- ¶ **Incorporation of prior knowledge:** Decision trees can integrate expert knowledge through the design of decision questions, improving classification performance.



Advantages of Decision Trees

- ¶ **Handles categorical data directly:** Decision trees do not require numeric representation of features.
- ¶ **Interpretable structure:** The tree provides clear classification rules that are easy to understand.
- ¶ **Flexibility:** Trees can handle both nominal and ordinal features.

Splitting Criteria

To build an effective tree, we need to decide **which feature to split on at each node**. Common criteria include:

- ¶ **Information Gain:** Measures the reduction in entropy (uncertainty) after a split. The feature that provides the highest information gain is chosen.
- ¶ **Gini Index:** Measures the impurity of a node. A lower Gini Index indicates a better split.
- ¶ **Chi-square Test:** Evaluates the statistical significance of splits for categorical features.

Overfitting and Pruning

Decision trees can grow very large and perfectly fit the training data, which may reduce their ability to generalize to new data. This is known as **overfitting**. To prevent this, trees can be **pruned**, either by limiting the depth of the tree or by removing nodes that provide little improvement in classification accuracy.

Decision trees are widely used in applications such as medical diagnosis, customer classification, and pattern recognition, making them a fundamental tool for nonmetric data analysis.

Information, Entropy, and Impurity

Information can be understood as the reduction of uncertainty in a system. Attributes that are highly informative are those that reduce uncertainty the most. The information content of a single event E is measured in **units of information** and is given by:

$$I(E) = \log \frac{1}{P(E)} = -\log P(E)$$

where $P(E)$ is the prior probability of the event occurring. Intuitively, events that are very likely carry little information, while rare or unexpected events carry more information.

For a system with only two possible outcomes (0 and 1), the logarithm is typically taken in base 2, making the unit of information a **bit**. If both events are equally likely [$P_1(E)=P_2(E)=\frac{1}{2}$], then the information content of each event is:

$$I(E_1)=I(E_2)=-\log_2(\frac{1}{2})=1$$

This means that one bit of information is conveyed when one of two equally likely events occurs. However, if the events have unequal probabilities, for example, $P_1(E)=\frac{1}{4}$ and $P_2(E)=\frac{3}{4}$, then, the less probable event conveys more information:

$$[I(E_1)=-\log_2(\frac{1}{4})=2] \text{ and } [I(E_2)=-\log_2(\frac{3}{4})=0.415].$$

Thus, the rarer the event, the higher its informational value.

Entropy is a measure of the **disorder** or **unpredictability** of a system. It is often used for discrete variables, whereas **variance** is used for continuous variables. In a binary classification problem with classes C_0 and C_1 , consider a set of examples S where the class distribution is (p_0, p_1) with $p_1=1-p_0$. The entropy $H(S)$ of the set is defined as the sum of the information of each class:

$$H(S) = -p_0 \log_2 p_0 - p_1 \log_2 p_1$$

Entropy reaches its maximum when the classes are equally likely, indicating maximum uncertainty, and is zero when all examples belong to a single class, indicating no uncertainty.

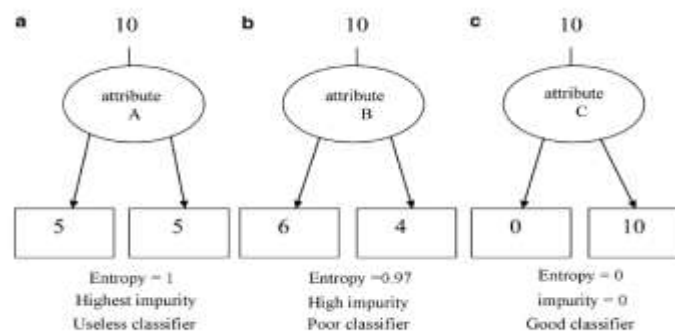


Fig. 3.3 Comparison of three decision nodes based on different attributes

Entropy measures the **uncertainty** or **impurity** of a set of examples at a decision node in a classification problem. An attribute that splits a dataset into classes with an even distribution, such as (0.5,0.5) (see Fig. 3.3a), has maximum entropy, equal to 1 bit, indicating **high uncertainty**. This makes it a poor choice for classification because it does not favor any particular class.

If an attribute splits the examples into (0.6,0.4), the entropy is calculated as:

$-0.6 \log_2 0.6 - 0.4 \log_2 0.4 = 0.97$ (see Fig. 3.3b). Here, the uncertainty is slightly lower, making it somewhat more useful.

If all examples for an attribute belong to the same class (i.e., a split of (0,1) or (1,0), as in Fig. 3.3c), the entropy is **zero**, indicating **no uncertainty**. Such an attribute provides perfect classification.

In general, entropy can be interpreted as a measure of **impurity** at a node. Lower impurity corresponds to a more skewed class distribution, which results in a more useful classification. For example:

- ¶ A node with distribution (0,1) has **zero impurity** and is a good classifier.
- ¶ A node with distribution (0.5,0.5) has **maximum impurity** and is not useful.

For a target attribute that can take c different values (a multi-way split), the entropy of a set S is given by:

$$H(p) = - \sum_{i=1}^c p_i \log_2 p_i$$

where p_i is the proportion of examples in class i . The maximum possible entropy for such an attribute is $\log_2 c$, representing the highest level of uncertainty.

Other measures of impurity, which can also guide the choice of the best split, include:

- ¶ **Gini Impurity:**

$$\text{Gini}(p) = 1 - \sum_i p_i^2$$

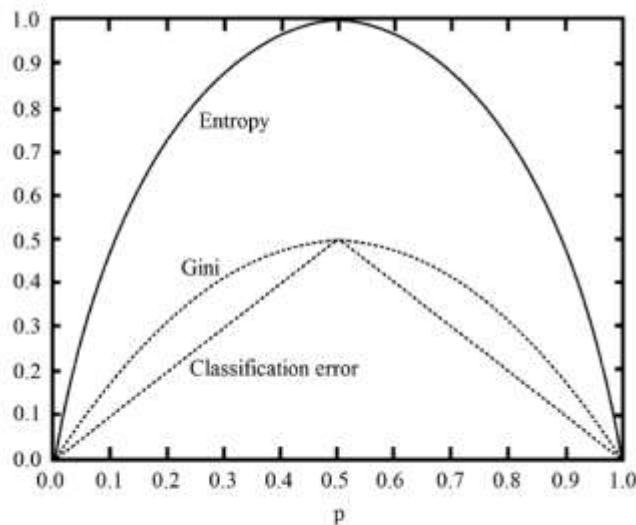


Fig. 3.4 Impurity measures for binary classification

$$\text{classification error}(p) = 1 - \max(p, 1-p)$$

The **Gini impurity** represents the expected error rate if the class label of an example is randomly chosen according to the class distribution at a node. In other words, it estimates how often a randomly selected item would be misclassified.

For **binary classification**, the values of common impurity measures— **entropy**, **Gini impurity**, and **classification error**— follow a similar pattern (see Fig. 3.4):

- ¶ All three measures reach their **maximum** when the classes are evenly distributed ($p=0.5$), reflecting **maximum uncertainty**.
- ¶ All three measures reach their **minimum** when all examples belong to a single class ($p=0$ or $p=1$), reflecting **zero uncertainty**.

A limitation of the **classification error** is that its derivative is **discontinuous**, which can make it less suitable when optimizing over a **continuous parameter space**, such as when searching for the best threshold for splitting data in decision tree algorithms.

- ¶ **Classification Error**, which similarly quantifies how mixed the classes are in a node.

Entropy and other impurity measures are fundamental in decision tree learning, helping identify the attributes that best separate the data into distinct classes.

Information Gain

In decision tree construction, one of the key challenges is to select the **best attribute** for splitting the data at each decision node. The node usually contains a **mixed set of instances**, and the goal is to find an attribute that separates these instances into **more homogeneous subsets** (see Fig. 3.5).

To measure how well an attribute splits the data, we use **information gain**. Information gain quantifies the **expected reduction in impurity** that results from partitioning the dataset according to a given attribute.

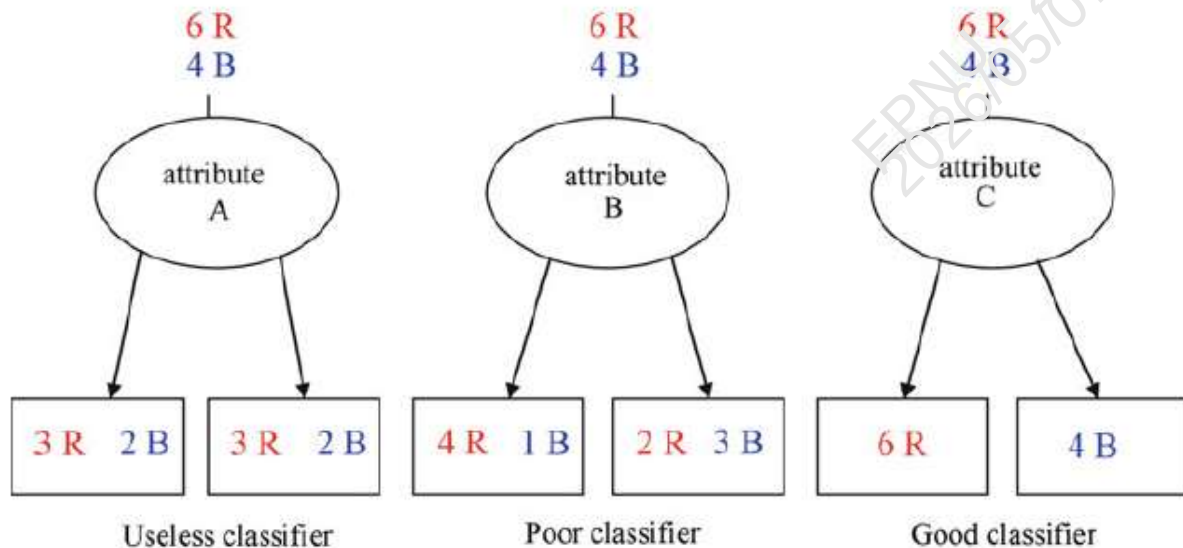


Fig. 3.5 Different attributes splitting a mixture of instances. Attribute C provides the purest split and is the best attribute to use for classification

This quantity, denoted as $\text{Gain}(S, A)$, measures how much the attribute A helps in organizing the data into more **homogeneous subsets**.

Mathematically, it is expressed as:

$$\text{Gain}(S, A) = \text{Impurity}(S) - \sum_{i=1}^k \frac{|S_{vi}|}{|S|} \text{Impurity}(S_{vi})$$

Suppose an attribute A has a set of possible values $\{v_1, v_2, v_3 \dots v_k\}$, and let $|S_{v_i}|$ denote the number of examples in the dataset S that have the value v_i for attribute A . The formula for **information gain** has two parts:

1. The **first term** represents the impurity of the original dataset S before any splitting.
2. The **second term** represents the expected impurity after S is divided according to the values of attribute A . This expected impurity is computed as the **sum of the impurities of each subset** $|S_{v_i}|$, weighted by the fraction of examples in that subset ($|S_{v_i}|/|S|$).

When **entropy** is used as the measure of impurity, this quantity is referred to as **information gain**. It quantifies how much uncertainty is reduced by splitting the data on attribute A , and higher gain indicates a more informative attribute.

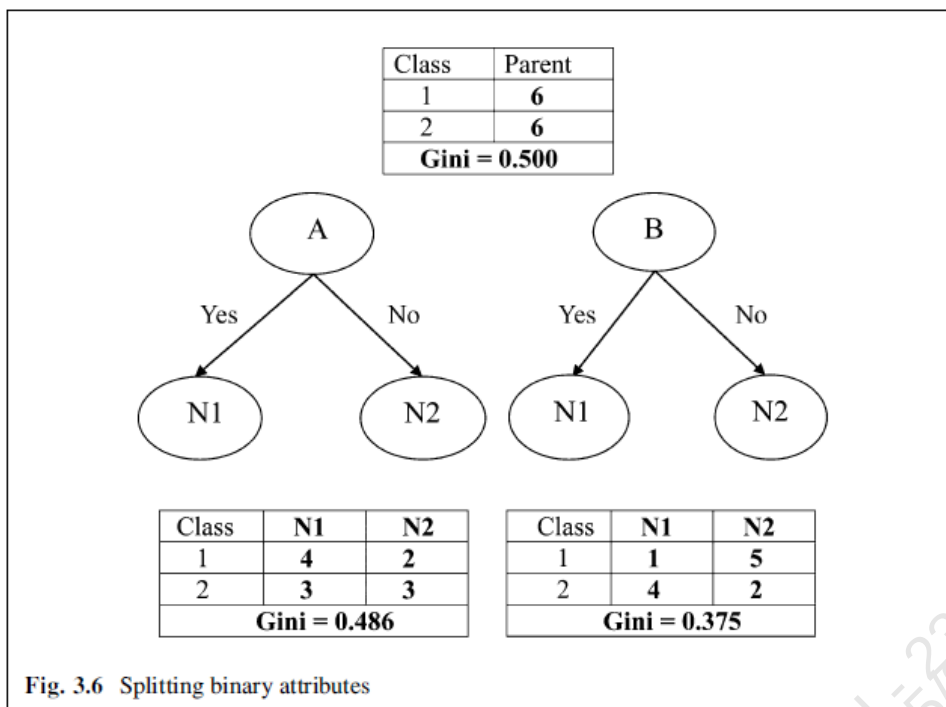
Example 3.1 Using the Gini index to find the gain

With reference to Fig. 3.6, there are two attributes, A and B , which can be used to split the data (comprising 12 instances) into smaller subsets. Before splitting (i.e., the parent node), the Gini index is 0.5 since there is an equal number of cases from both classes.

If attribute A is chosen to split the data, the Gini index for node N_1 is 0.4898 (i.e., $1 - [(4/7)^2 + (3/7)^2] = 24/49$) and for node N_2 it is 0.480 (i.e., $1 - [(2/5)^2 + (3/5)^2] = 12/25$). The weighted average for the two descendant nodes is $(7/12) \times 0.4898 + (5/12) \times 0.480 = 0.486$.

Similarly, if we use attribute B , the weighted average of the Gini index for attribute B is 0.375.

Since the subsets for attribute B have a smaller Gini index (i.e., smaller impurity), it is preferred to attribute A . [Or the gain in using attribute B is larger ($0.5 - 0.375 = 0.125$) than the gain in using attribute A ($0.5 - 0.486 = 0.014$).]



Example 3.2 Using the ID3 algorithm to build a decision tree.
 Suppose we want to train a decision tree using the examples (instances) in
 Table 3.1 Examples of decisions made over the past ten weekends

Example	Weather	Parents Visiting?	Money	Decision (Category)
1	Sunny	Yes	Rich	Cinema
2	Sunny	No	Rich	Tennis
3	Windy	Yes	Rich	Cinema
4	Rainy	Yes	Poor	Cinema
5	Rainy	No	Rich	Stay in
6	Rainy	Yes	Poor	Cinema
7	Windy	No	Poor	Cinema
8	Windy	No	Rich	Shopping
9	Windy	Yes	Rich	Cinema
10	Sunny	No	Rich	Tennis

The first thing is to find the attribute for the root node. To do this, we need to calculate the entropy, $H(S)$, before any splitting. Using (3.3), this comes to $S = 1.571$ [viz., $-0.6 \log_2 0.6 - 0.2 \log_2 0.2 - 2 \times (0.1 \log_2 0.1)$].

Then we need to determine the values of $\text{Gain}(S, \text{parents})$, $\text{Gain}(S, \text{weather})$, and $\text{Gain}(S, \text{money})$, using (3.6):

$$\begin{aligned}\text{Gain}(S, \text{parents}) &= 1.571 - (|S_{\text{yes}}|/10) \times \text{Entropy}(S_{\text{yes}}) - (|S_{\text{no}}|/10) \\ &\quad \times \text{Entropy}(S_{\text{no}}) = 1.571 - (0.5) \times 0 - (0.5) \times (1.922) = 0.61\end{aligned}$$

If “parents coming?” is the node, then five instances will go down the “Yes” branch (and then will all be class “cinema,” with an entropy of zero: this would be a terminal node) and five will go down the “No” branch [and they will comprise two “tennis,” one “stay-in,” one “cinema,” and one “shopping”: the entropy of this node is $-0.4 \log_2 0.4 - 3 \times (0.2 \log_2 0.2)$, which is 1.922].

$$\begin{aligned}\text{Gain}(S, \text{weather}) &= 1.571 - (|S_{\text{sun}}|/10) \times \text{Entropy}(S_{\text{sun}}) - (|S_{\text{wind}}|/10) \\ &\quad \times \text{Entropy}(S_{\text{wind}}) - (|S_{\text{rain}}|/10) \times \text{Entropy}(S_{\text{rain}}) \\ &= 1.571 - (0.3) \times (0.918) - (0.4) \times (0.8113) - (0.3) \\ &\quad \times (0.918) = 0.70\end{aligned}$$

$$\begin{aligned}\text{Gain}(S, \text{money}) &= 1.571 - (|S_{\text{rich}}|/10) \times \text{Entropy}(S_{\text{rich}}) \\ &\quad - (|S_{\text{poor}}|/10) \times \text{Entropy}(S_{\text{poor}}) \\ &= 1.571 - (0.7) \times (1.842) - (0.3) \times 0 = 0.2816\end{aligned}$$

This means that the weather attribute, with its three branches, should be the first (root) node (Fig. 3.7a).

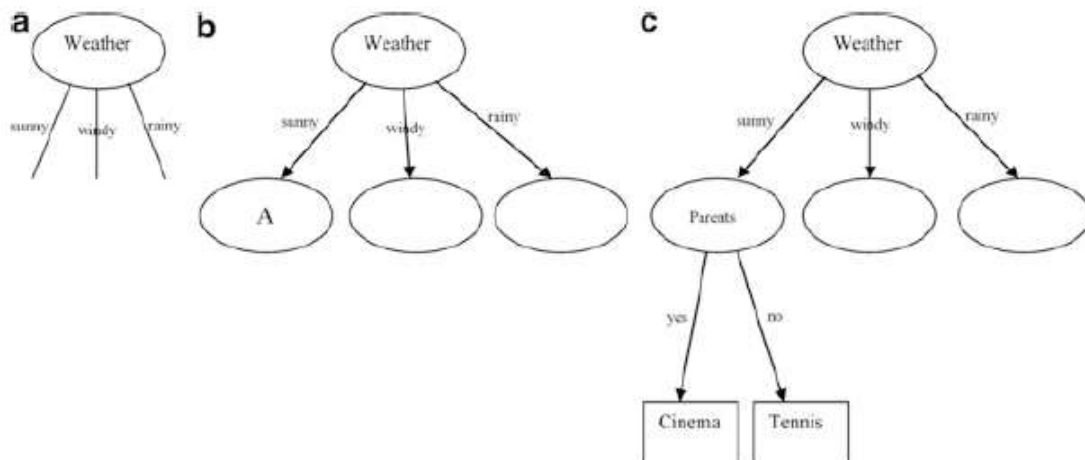
Now we look at the branches. For the sunny branch, $S_{\text{sunny}} = \{1, 2, 10\}$. Since the classes (cinema, tennis, and tennis respectively) are not the same, we will need another decision node here (Fig. 3.7b). The same situation occurs for the other two branches (where $S_{\text{windy}} = \{3, 7, 8, 9\}$, and $S_{\text{rainy}} = \{4, 5, 6\}$).

Returning to the sunny branch, we are only interested in the three examples $\{1, 2, 10\}$ and we set S to be S_{sunny} , from which $H(S)$ turns out to be 0.918 (viz., $-0.1 \log_2 0.1 - 0.2 \log_2 0.2$), since two examples end up together (as “tennis”) and one ends up on its own (as “cinema”). We now need to calculate Gain(S_{sunny} , parents) and Gain(S_{sunny} , money):

$$\begin{aligned} \text{Gain}(S_{\text{sunny}}, \text{parents}) &= 0.918 - (|S_{\text{yes}}|/|S|) \times \text{Entropy}(S_{\text{yes}}) - (|S_{\text{no}}|/|S|) \\ &\quad \times \text{Entropy}(S_{\text{no}}) \\ &= 0.918 - (1/3) \times 0 - (2/3) \times 0 = 0.918 \end{aligned}$$

Note that $\text{Entropy}(S_{\text{yes}})$ and $\text{Entropy}(S_{\text{no}})$ are both zero, because both S_{yes} and S_{no} contain examples which are all in the same category (cinema and tennis, respectively). Hence, the parents attribute should be chosen next. It has two branches for “Yes” and “No”: the “Yes” branch contains a single example $\{1\}$, and the “No” branch contains two examples, $\{2, 10\}$ which are in the same class. Hence both these branches end at leaf nodes (Fig. 3.7c).

The rest of the tree is left as an exercise for the reader!



Questions

1. What is meant by a **homogeneous subset** in decision trees?
2. Why is entropy measured in bits?
3. What is the difference between **Gini impurity** and **classification error**?
4. Why do we weight child nodes when calculating gain?
5. What is a decision boundary?
6. What measure does ID3 use to select the best attribute?
7. What is entropy in the context of decision trees?
8. What is information gain?
9. Why is the attribute with the highest information gain selected first?
10. If all examples result in the same decision, what would be the entropy?
11. Which attribute do you think would be selected as the root node? Why?
12. If Weather = Sunny and Parents Visiting = No, what is the most likely decision?
13. If Weather = Rainy and Parents Visiting = Yes, what is the decision?
14. What happens if a new example has attribute values not seen before in training?
15. Build the first level of the decision tree manually.
16. Calculate the entropy of the full dataset.
17. Calculate the information gain for the attribute Weather.
18. Draw the complete decision tree.
19. How would the tree change if more training examples were added?

Chapter 4: Supervised Learning

4.1 Parametric and Non-parametric Learning

Supervised learning methods in statistical classification are generally divided into **parametric** and **non-parametric** approaches.

Parametric methods assume knowledge of the **probability distributions** of the classes. They estimate parameters such as the **mean** and **standard deviation** to create a compact representation of each class. Examples of parametric methods include:

- ¶ **Bayes' decision**, which is explicitly based on probability distributions.
- ¶ **Discriminant analysis**, which separates classes using mathematical functions.

Parametric methods usually take **longer to train**, but once trained, they classify new test data **quickly**.

When the underlying probability distributions are unknown, **non-parametric methods** are used. These methods either:

1. **Estimate the density functions** (e.g., the **Parzen window approach**).
2. **Directly construct decision boundaries** from training data without using probabilities (e.g., the **k-nearest neighbor rule**).

A **multilayer perceptron** can also be considered a **non-parametric supervised method**, as it constructs decision boundaries from the data rather than relying on probability distributions.

4.2 Parametric Learning

Parametric learning focuses on methods that assume a known probability distribution and estimate its parameters to classify data.

4.2.1 Bayesian Decision Theory

Bayesian decision theory provides a framework for making **optimal decisions under uncertainty**. It combines prior knowledge of class probabilities with observed measurements to calculate **posterior probabilities**.

4.2.1.1 Single Feature (1D)

In statistical pattern recognition, classifiers are designed to use all available information, including measurements and **a priori (prior) probabilities**. By combining this information, we calculate **posterior probabilities** (or **measurement-conditioned probabilities**).

From these posterior probabilities, we can formulate a **decision rule** to classify new data points in the most optimal way.

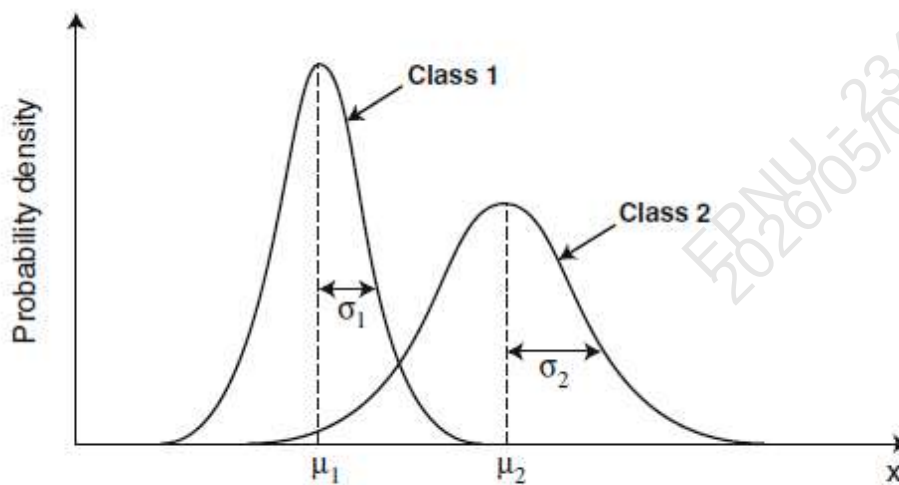


Fig. 5.1 Probability density functions for two classes, 1 and 2; often they will be Gaussian in shape

Two-Class Case with Known Prior Probabilities

Let us assume there are only two classes:

Class 1 (x_1)

Class 2 (x_2)

Suppose the prior probabilities are known:

$$P(x_1) = 0.7$$

$$P(x_2) = 0.3$$

These probabilities sum to 1:

$$P(x_1) + P(x_2) = 1$$

If we are given a new sample and have no additional information, the decision rule is to assign the sample to the class with the larger prior probability.

Since: $P(x_1) > P(x_2)$, it classify the sample as belonging to x_1 .

Probability of Classification Error, In general, the probability of error is:

$$P(\text{error}) = P(\text{choose } x_2 \mid x_1) \checkmark P(x_1) + P(\text{choose } x_1 \mid x_2) \checkmark P(x_2)$$

This means:

- ¶ The probability of choosing class 2 when the true class is 1
- ¶ Plus the probability of choosing class 1 when the true class is 2

In this example, since we always choose x_1 :

$$P(\text{error}) = P(\text{choose } x_1 \mid x_2) \checkmark P(x_2)$$

All samples from class 2 will be misclassified. Therefore:

$$P(\text{error}) = 1 \times 0.3$$

$$P(\text{error}) = 0.3$$

So the probability of classification error is 0.3 (30%).

Single Feature Case (1D)

Now suppose we measure a single feature x (for example, length, brightness, or weight).

From the training data, we estimate the probability density functions:

$$p(x \mid x_1)$$

$$p(x \mid x_2)$$

These are called class-conditional probability densities. They represent:

- ¶ The probability of measuring value x given the object belongs to class x_1
- ¶ The probability of measuring value x given the object belongs to class x_2

If we have many training samples, these distributions are often Gaussian (normal) in shape due to the Central Limit Theorem.

Classification Using Bayes's Rule

Given a new sample with feature value x , we want to compute the posterior probability:

$$P(x_i | x)$$

This represents the probability that the sample belongs to class x_i given the observed feature value x .

Using Bayes's Rule:

$$P(x_i | x) = [p(x | x_i) \cdot P(x_i)] / p(x)$$

Where:

- ¶ $p(x | x_i)$ is the likelihood
- ¶ $P(x_i)$ is the prior probability
- ¶ $p(x)$ is the evidence (normalizing constant)

Decision Rule

We calculate:

$$P(x_1 | x)$$

$$P(x_2 | x)$$

Then we assign the sample to the class with the larger posterior probability.

This is known as the Bayes Decision Rule, and it minimizes the probability of classification error.

Figure 5.1 shows the probability density functions for two classes, Class 1 and Class 2. These distributions are often Gaussian (normal) in shape, characterized by:

- ¶ Mean values: μ_1 and μ_2
- ¶ Standard deviations: σ_1 and σ_2

The horizontal axis represents the feature x , and the vertical axis represents the probability density.

Maximum a Posteriori (MAP) Classifier

Using Bayes's rule, the posterior probability for each class ($i = 1, 2$) given a particular feature value x is:

$$P(x_i | x) = [p(x | x_i) \cdot P(x_i)] / p(x) \quad (5.2)$$

This classifier is called a **Maximum A Posteriori (MAP) classifier**, because we assign the sample to the class with the maximum posterior probability.

Decision Rule

Our decision rule becomes:

If

$$[p(x | x_1) \cdot P(x_1)] / p(x) > [p(x | x_2) \cdot P(x_2)] / p(x)$$

choose x_1

else

choose x_2

Note:

- ¶ Upper-case P denotes probability.
- ¶ Lower-case p denotes probability density.

The term $p(x)$ represents how frequently we observe feature value x . It ensures that the posterior probabilities sum to 1. However, since it appears in both sides of the inequality, it does not affect the final decision.

Simplified Decision Rule

Since $p(x)$ appears in both numerators, we can divide both sides by $p(x)$. The decision rule simplifies to:

If $p(x | \gamma_1) / p(x | \gamma_2) > P(\gamma_2) / P(\gamma_1)$
choose γ_1

else
choose γ_2

Likelihood Ratio Test

The quantity:

$$p(x | \gamma_1) / p(x | \gamma_2)$$

is called the **likelihood ratio**.

This decision method is known as the **Likelihood Ratio Test**.

It compares:

- ¶ The evidence provided by the data (likelihood ratio),
with
- ¶ The prior probabilities of the classes.

If the likelihood ratio exceeds the threshold $P(\gamma_2) / P(\gamma_1)$, we choose Class 1;
otherwise, we choose Class 2.

Example 5.1

Suppose we have a single feature x whose class-conditional densities are Gaussian.

Assume:

- ¶ Equal variances: $\sigma^2 = 1$

- ¶ Different means: $\mu_1 = 4$ and $\mu_2 = 10$
- ¶ Equal prior probabilities:
 $P(x_1) = 0.5$
 $P(x_2) = 0.5$

The class-conditional probability density functions and the corresponding posterior probabilities are illustrated in Figure 5.2.

Using 1D Gaussian Class-Conditional Densities

For a one-dimensional Gaussian distribution, the probability density function is:

$$p(x | x_i) = (1 / \sqrt{2\pi}\sigma_i) \exp[- (x - \mu_i)^2 / (2\sigma_i^2)]$$

Since $\sigma_i = 1$, this simplifies to:

$$p(x | x_i) = (1 / \sqrt{2\pi}) \exp[- (x - \mu_i)^2 / 2]$$

Because both classes have equal variance and equal priors, the likelihood ratio test becomes:

$$\frac{\exp[- (x - 4)^2 / 2]}{\exp[- (x - 10)^2 / 2]} > 0.5 / 0.5$$

Since:

$$0.5 / 0.5 = 1$$

The decision rule becomes:

If

$$\frac{\exp[- (x - 4)^2 / 2]}{\exp[- (x - 10)^2 / 2]} > 1$$

choose x_1

else

choose x_2

Simplifying the Decision Rule

Taking natural logarithms of both sides simplifies the inequality:

$$\ln(x + 4) / 2 > \ln(x + 10) / 2$$

Multiplying both sides by $\ln 2$ (which reverses the inequality sign):

$$(x + 4) \ln 2 < (x + 10) \ln 2$$

Expanding both sides:

$$x \ln 2 + 4 \ln 2 < x \ln 2 + 10 \ln 2$$

Cancel $x \ln 2$ from both sides:

$$4 \ln 2 < 10 \ln 2$$

Rearranging:

$$4 < 10$$

$$x < 7$$

Final Decision Rule

If $x < 7$, choose x_1

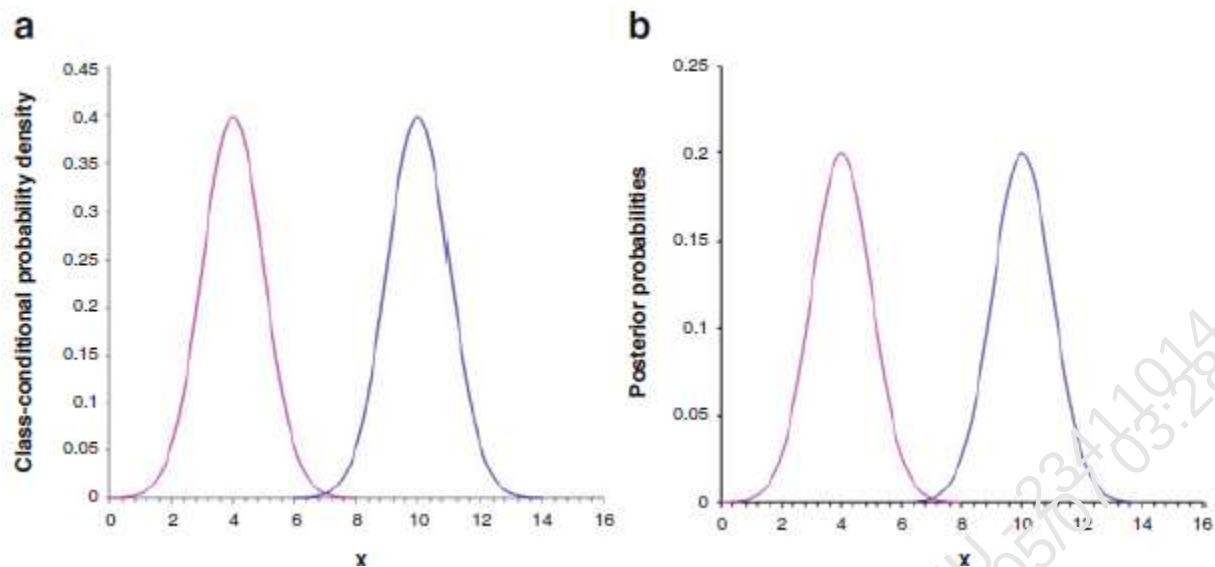
If $x > 7$, choose x_2

Thus, the decision boundary occurs at:

$$x = 7$$

This result makes intuitive sense because 7 is exactly midway between the two means (4 and 10), given that:

- ¶ Variances are equal
- ¶ Prior probabilities are equal



By cross-multiplying the terms, applying the natural logarithm, and simplifying the resulting expression, we obtain:

$$(x - 4) \ln \frac{1}{(x - 10)} < 0$$

Solving this inequality gives the decision rule:

If $x < 7$, choose x_1

Otherwise, choose x_2

As expected, the decision boundary lies exactly halfway between the two mean values. Since the means are 4 and 10, the midpoint is 7.

The decision threshold represents the point at which the posterior probabilities of the two classes are equal. In other words, it is the value of x where:

$$P(x_1 | x) = P(x_2 | x)$$

Graphically, this corresponds to the intersection of the two posterior probability curves. When the prior probabilities are equal, this intersection point is also where the class-conditional probability density functions cross.

It is important to note that the total area under each class-conditional probability density function equals 1, since each represents a valid probability distribution. For posterior probabilities, however, it is their sum at any given value of x that equals 1.

Example 5.2

Assume that the prior probabilities are equal:

$$P(x_1) = P(x_2) = 0.5$$

The means remain the same as in Example 5.1:

$$\mu_1 = 4$$

$$\mu_2 = 10$$

However, the class-conditional densities now have different variances.

$$\sigma_1^2 = 4$$

$$\sigma_2^2 = 1$$

The corresponding probability density functions are illustrated in Figure 5.3.

Likelihood Ratio Test

For a one-dimensional Gaussian distribution:

$$p(x | x_i) = (1 / \sqrt{2\pi\sigma_i^2}) \exp[- (x - \mu_i)^2 / (2\sigma_i^2)]$$

Substituting the given values:

For Class 1 ($\sigma_1^2 = 4$):

$$p(x | x_1) = (1 / \sqrt{8\pi}) \exp[- (x - 4)^2 / 8]$$

For Class 2 ($\sigma_2^2 = 1$):

$$p(x | x_2) = (1 / \sqrt{2\pi}) \exp[- (x - 10)^2 / 2]$$

Using the likelihood ratio test and equal priors, the decision rule becomes:

$$\frac{(1/2) \exp[-(x-4)^2/8]}{\exp[-(x-10)^2/2]} > 1$$

If the inequality holds, choose x_1 ; otherwise, choose x_2 .

Important Observation

Unlike Example 5.1 (where variances were equal), the decision boundary is no longer simply the midpoint between the two means.

Because the variances are different:

- ¶ Class 1 has a wider (more spread out) distribution
- ¶ Class 2 has a narrower (more concentrated) distribution

As a result, the decision boundary shifts toward the class with the larger variance.

In this case, solving the inequality leads to a quadratic equation, meaning there may be one or even two decision boundaries depending on the parameter values.

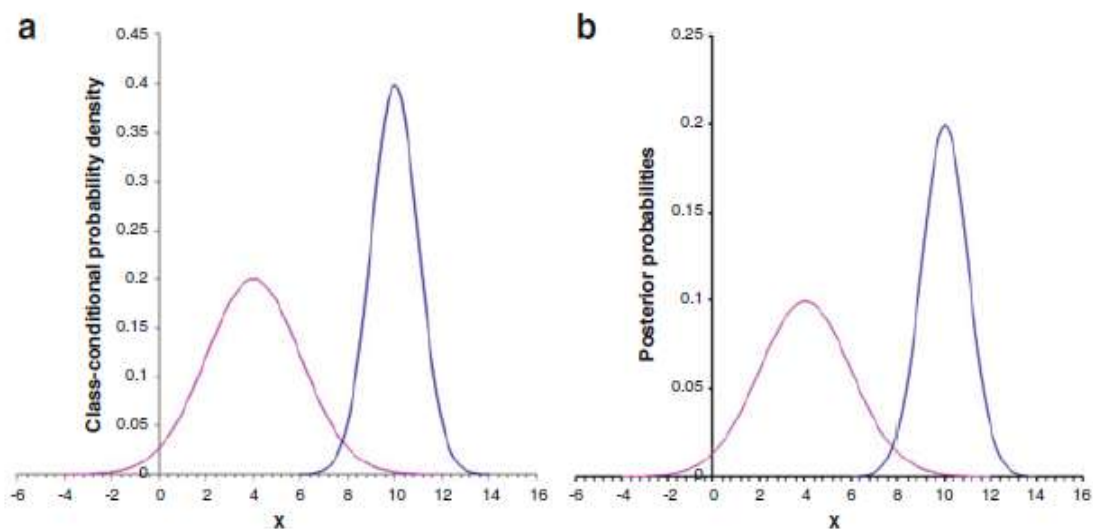


Fig. 5.3 For Gaussians with means of $\mu_1 = 4$ (in red) and $\mu_2 = 10$ (in blue), and variances of $\sigma_1^2 = 4$ and $\sigma_2^2 = 1$ (a) the class-conditional probabilities and (b) the posterior probabilities of measuring a feature, x

By cross-multiplying, taking natural logarithms, and simplifying the inequality, we obtain:

$$8 \ln(1/2) + (x - 4) > 4(x - 10)$$

Rearranging the terms leads to the quadratic expression:

$$3x - 72x + (384 + 8 \ln(1/2)) > 0$$

Solving this quadratic equation produces two distinct roots, which correspond to two separate decision thresholds:

$$x = (72 \pm 25.3) / 6$$

This gives:

$$x = 7.78 \text{ and } x = 16.22$$

Interpretation of the Results

The class-conditional probability density functions and the posterior probabilities are illustrated in Figure 5.3. Because the prior probabilities are equal, the posterior probabilities are simply scaled versions of the class-conditional densities.

The lower decision threshold ($x = 7.78$) is clearly visible in the figure. However, the second decision threshold ($x = 16.22$) is less obvious.

The reason for this second intersection is that the posterior probability for class x_1 decreases more slowly than that for class x_2 as x becomes large. Since class x_1 has the larger variance, its distribution has heavier tails. As a result, the two posterior probability curves intersect again at a higher value of x .

Final Decision Rule

The classification rule becomes:

If $x < 7.78$ or $x > 16.22$, choose x_1

If $7.78 < x < 16.22$, choose x_2

Example 5.3

Now suppose the prior probabilities are unequal:

$$P(x_1) = 2/3$$

$$P(x_2) = 1/3$$

The variances remain equal:

$$\sigma_1^2 = \sigma_2^2 = 1$$

Using the likelihood ratio test, the decision rule becomes:

$$\frac{[\exp(-(x-4)^2/2)]^{1/2} (2/3)}{[\exp(-(x-10)^2/2)]^{1/2} (1/3)} > 1$$

If the inequality holds, choose x_1 ; otherwise, choose x_2 .

Solving the Decision Rule

After simplifying the expression and solving the resulting quadratic equation, we obtain a single decision boundary at:

$$x = 7.12$$

Unlike Example 5.2, only one threshold appears because the quadratic equation has equal roots.

Interpretation

The decision boundary corresponds to the point where the posterior probabilities are equal:

$$P(x_1 | x) = P(x_2 | x)$$

Since the prior probabilities are not equal in this case, the decision boundary does not occur at the midpoint between the two means.

It is important to note that when prior probabilities are unequal, the decision threshold does not coincide with the intersection of the class-conditional probability density functions. Instead, it corresponds to the intersection of the posterior probability curves.

Because class x_1 has a higher prior probability ($2/3$), the decision boundary shifts toward class x_2 . This reflects the increased likelihood of assigning samples to x_1 .

Extension to Higher Dimensions

When multiple features are involved:

- ¶ In two dimensions, decision boundaries become lines.
- ¶ In three dimensions, they become planes.
- ¶ In higher dimensions, they become hyperplanes.

Thus, Bayesian classification generalizes naturally from 1D thresholds to multidimensional decision surfaces.

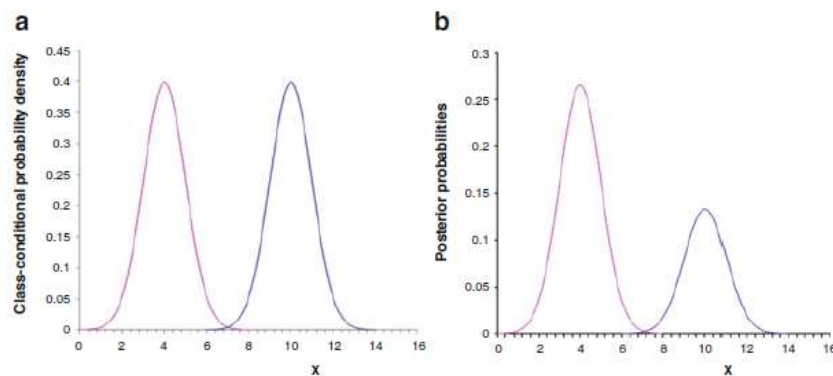


Fig. 5.4 For Gaussians with means of $\mu_1 = 4$ and $\mu_2 = 10$, equal variances ($\sigma_1^2 = \sigma_2^2 = 1$) and prior probabilities $P(\omega_1) = 2/3$ and $P(\omega_2) = 1/3$ (a) the class-conditional probabilities and (b) the posterior probabilities of measuring a feature, x

Example 5.4

Select the optimal decision where $p(x|\omega_1) = N(2, 0.25)$, $p(x|\omega_2) = N(1.5, 0.04)$; $P(\omega_1) = 2/3$, $P(\omega_2) = 1/3$; and $\lambda = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix}$.
Since $\lambda_{21} > \lambda_{11}$, decide ω_1 , if

$$\frac{p(x|\omega_1)}{p(x|\omega_2)} > \frac{\lambda_{12} - \lambda_{22} \cdot P(\omega_2)}{\lambda_{21} - \lambda_{11} \cdot P(\omega_1)}$$

$$p(x|\omega_1) = \frac{2}{\sqrt{2\pi}} e^{-2(x-2)^2} \quad \text{and} \quad p(x|\omega_2) = \frac{5}{\sqrt{2\pi}} e^{-\frac{25}{2}(x-\frac{3}{2})^2}$$

The likelihood ratio for the two normal distributions is

$$\frac{p(x|\omega_1)}{p(x|\omega_2)} = \frac{2}{5} e^{-2(x-2)^2 + \frac{25}{2}(x-1.5)^2} > \frac{1-0}{2-0} \cdot \frac{1/3}{2/3} = \frac{1}{4}$$

$$e^{-2(x-2)^2 + \frac{25}{2}(x-1.5)^2} > \frac{5}{8}$$

taking (natural) logs $\Rightarrow -2(x-2)^2 + 25(x-1.5)^2/2 > -0.47$

- $x^2 - 2.8095x + 1.9614 > 0$
- $x = 1.514$ or $x = 1.296$

Therefore, $x > 1.514$ or $x < 1.296$ to decide ω_1 ;
and if $1.296 < x < 1.514$, decide ω_2 .

Questions

1. What is the difference between **parametric** and **non-parametric** supervised learning methods?
2. Give two examples of parametric supervised learning methods.
3. What is Bayesian decision theory, and why is it useful in classification?
4. How are **prior probabilities** used in Bayesian decision making?
5. What is a **posterior probability**, and how is it calculated?
6. Write Bayes's Rule formula for a two-class problem and explain each term.
7. If $P(x_1) = 0.7$ and $P(x_2) = 0.3$, what class would you assign a sample if no features are known?
8. How do you calculate the **probability of classification error** in a two-class case with known priors?
9. What are **class-conditional probability densities**, and what do they represent?
10. Why are class-conditional densities often assumed to be Gaussian?
11. Explain how you would classify a new sample using a single feature x .
12. What is the difference between **likelihood**, **prior probability**, and **evidence** in Bayes's Rule?
13. What does the **MAP classifier** do?
14. How does the MAP classifier relate to the Bayes Decision Rule?
15. Given a new measurement x , explain step by step how the MAP classifier decides which class the sample belongs to.
16. How would the decision boundary change if the prior probabilities $P(x_1)$ and $P(x_2)$ were equal?
17. If the variances of the two Gaussian distributions are different, how does this affect the Bayes decision boundary?
18. How could you extend the MAP classifier to more than two classes?

Chapter 5: Unsupervised Learning

5.1 Clustering

Unsupervised learning is a type of machine learning where the class labels of the data are unknown. The goal is to explore the data and see whether it naturally forms clusters – groups of similar objects.

Cluster analysis divides data into clusters (or classes) that are ideally meaningful and useful. These clusters may or may not align with human perception of similarity.

Applications of Clustering:

- ¶ Biology: To find groups of genes with similar functions.
- ¶ Climatology: To identify clusters of atmospheric pressure that affect climate.
- ¶ Medicine: To detect different variants of a disease.
- ¶ Business: To segment customers for targeted marketing.
- ¶ Information retrieval: To group search results into clusters and subclusters hierarchically.
- ¶ Imaging:
 - Segment images into different regions.
 - Compress image and video data by representing objects with cluster prototypes (vector quantization).

Principles of Clustering:

Clusters should:

1. Contain objects that are similar to each other.
2. Be different from objects in other clusters.

To achieve this, we use a (dis)similarity measure, often called a proximity measure.

Examples include:

- ¶ L1 norm (Manhattan distance / city-block distance): $d = |x_1 - y_1| + |x_2 - y_2|$
- ¶ L2 norm (Euclidean distance): $d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$
- ¶ Chebyshev distance (L_∞ norm / chessboard distance): $d = \max\{|x_1 - y_1|, |x_2 - y_2|\}$

Clustering Criterion: Sum of Squared Errors (SSE):

Once a (dis)similarity measure is chosen, we define a criterion function to optimize.

¶ The most commonly used criterion is the Sum of Squared Errors (SSE).

How SSE works:

1. Compute the distance of each data point to the closest cluster centroid.
2. Square this distance.
3. Sum over all data points to get the total error.

Clustering methods that minimize SSE are called minimum variance methods.

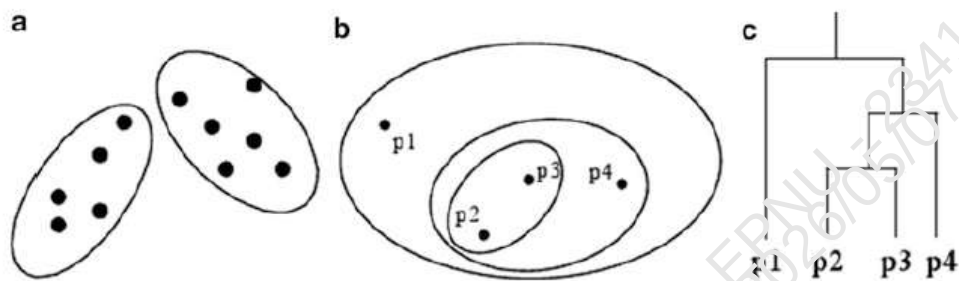


Fig. 8.1 (a) Partitional clustering (b) hierarchical clustering, represented as (c) a dendrogram

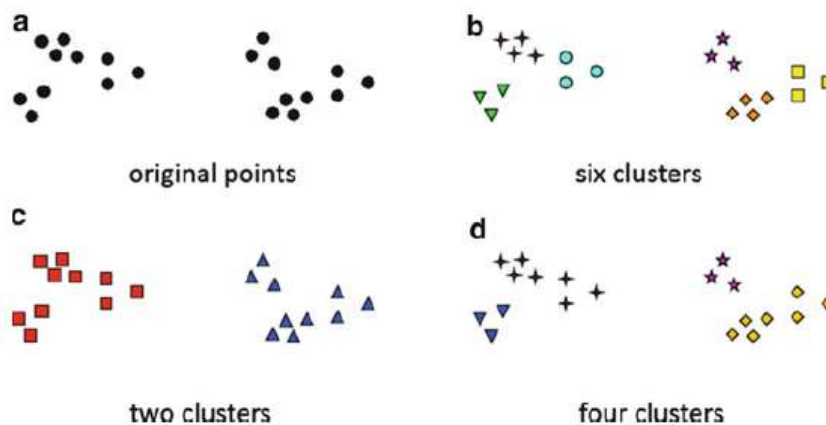


Fig. 8.2 Different ways of clustering the same set of points

Clustering methods that minimize SSE are called minimum variance methods.

Cluster Types and Considerations:

- ¶ Clusters can be based on **scatter matrices** used in Linear Discriminant Analysis (LDA).
- ¶ **Partitional (Flat) Clusters:** Data are divided into non-overlapping subsets such that each data point belongs to exactly one cluster.
- ¶ **Hierarchical Clusters:** Clusters are nested and often represented by a hierarchical tree or dendrogram.
- ¶ The definition of a cluster is subjective and often in the eye of the beholder; a dataset may appear to have different numbers of clusters depending on interpretation.
- ¶ **Non-Exclusive Clusters:** Data points may belong to multiple clusters.
- ¶ **Fuzzy Clustering:** Each data point belongs to every cluster with a weight (or probability) between 0 and 1.
- ¶ Because clusters are not well defined, it can be difficult to determine how far a clustering algorithm should continue.

Example Techniques:

- ¶ **Partitional Clustering:** k-means clustering and its variants.
- ¶ **Hierarchical Clustering:** Agglomerative hierarchical clustering

5.2 Types of Clustering Algorithms

Clustering algorithms can be broadly categorized based on how they form clusters:

1. Partitional Clustering Algorithms:

- Divide data into non-overlapping subsets.
- Each data point belongs to exactly one cluster.
- Examples: k-means, k-medoids, and their variants.

2. Hierarchical Clustering Algorithms:

- Form nested clusters that can be visualized as a tree or dendrogram.
- Can be **agglomerative** (bottom-up) or **divisive** (top-down).
- Examples: Agglomerative hierarchical clustering, divisive hierarchical clustering.

3. Density-Based Clustering Algorithms:

- Identify clusters as areas of high density separated by low-density regions.
- Can handle arbitrary-shaped clusters and noise.
- Examples: DBSCAN, OPTICS.

4. Model-Based Clustering Algorithms:

- Assume that data are generated from a mixture of underlying probability distributions.
- Use statistical models to assign probabilities of cluster membership.
- Examples: Gaussian Mixture Models (GMM), EM-based clustering.

5. Fuzzy Clustering Algorithms:

- Each data point can belong to multiple clusters with varying degrees of membership.
- Examples: Fuzzy C-Means (FCM).

5.3 k-Means Clustering

k-Means clustering (MacQueen 1967) is one of the oldest and most widely used clustering algorithms. It is a **partitional clustering** approach. Each cluster is represented by one prototype object, and a new data sample is assigned to the nearest prototype and therefore to that cluster.

Training Algorithm:

1. Randomly choose k objects from the training set as the initial prototypes.
2. Assign all other objects to the nearest prototype to form clusters (measured by Euclidean distance or another norm).
3. Update each prototype as the centroid of all objects assigned to that cluster.
4. Repeat step 2 until convergence (no data point changes clusters, or centroids remain the same). A weaker stopping condition may be used, e.g., until only 1% of points change clusters.

The **assignment step** is also called the **expectation step**, and the **update step** is the **maximization step**, making k-means a variant of the generalized Expectation-Maximization (EM) algorithm.

As a heuristic algorithm, k-means **does not guarantee a global optimum**. The result is sensitive to the initial choice of cluster centers. Larger datasets (~200 to 100,000

observations) work best. It is common to run the algorithm multiple times with different initial conditions.

The algorithm is essentially a **greedy approach** to partition n samples into k clusters to minimize an

$$\text{SSE} = \sum_{i=1}^K \sum_{x \in C_i} \text{dist}(c_i, x)^2$$

objective function, the **sum of squared errors (SSE)**: where c_i is the center of the i th cluster, and dist is the Euclidean distance (standardized features make clusters more circular).

Multiple Runs and Complexity:

- ¶ For two runs with the same k but different starting prototypes, choose the one with the **smallest SSE**.
- ¶ Algorithm complexity: $O(n \times k \times I \times d)$, where n = number of data points, k = clusters, I = iterations, d = features.

Pre-processing and Post-processing:

- ¶ **Pre-processing:** Standardize/normalize data; reduce outliers (median can replace mean to avoid centroids being pulled by outliers).
- ¶ **Post-processing:** Split loose clusters (high SSE) and merge close clusters (low SSE).

Bisecting k-Means Algorithm:

- ¶ Minimizes dependence on initial prototypes.
- ¶ Split points into two clusters, then recursively split each cluster.
- ¶ If $k = 2^m$, iterate m times. For k not a power of 2, use the nearest lower power of 2 and subcluster remaining clusters as needed.

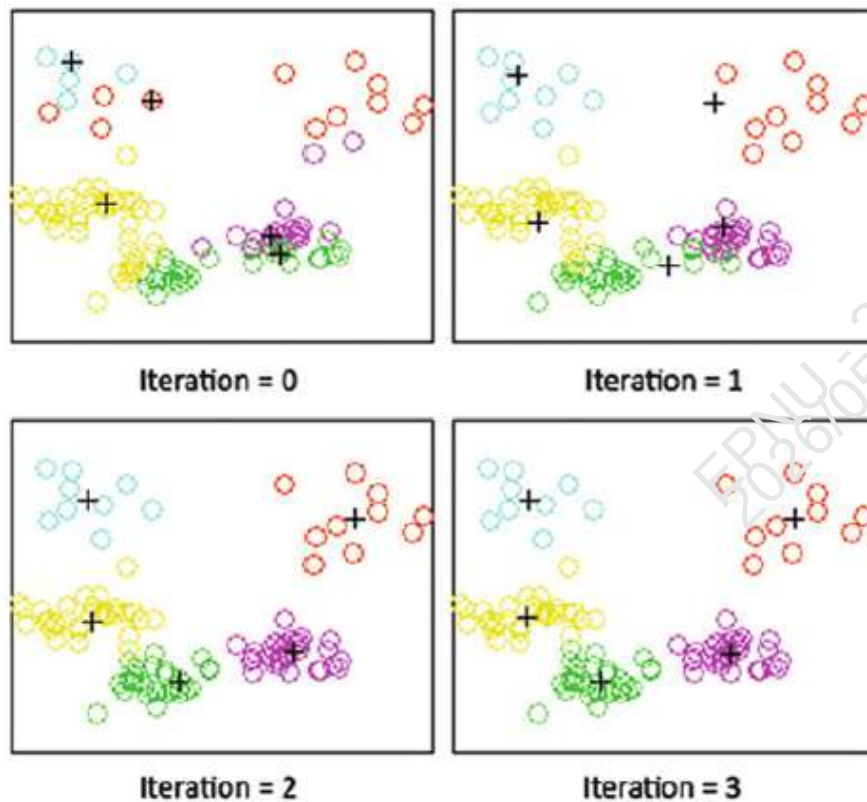


Fig. 8.3 Results of k -means clustering in various iterations (The prototypes in each iteration are marked with a “plus.”)

Exercise 1:

Cluster the following eight points into three clusters using the K-Means Algorithm.

Distance Function:

Use Manhattan Distance:

$$P(a, b) = |x_2 - x_1| + |y_2 - y_1|$$

Points:

Point	Coordinates
B1	(3, 9)
B2	(4, 8)
B3	(10, 10)
B4	(9, 8)
B5	(2, 3)
B6	(3, 2)
B7	(8, 9)
B8	(1, 4)

Initial Cluster Centers:

$C1 = B1 (3, 9)$

$C2 = B3 (10, 10)$

$C3 = B5 (2, 3)$

Required:

1. Perform Iteration 1 (assignment + new centers).
2. Perform Iteration 2.
3. Find the cluster centers after the second iteration.

Iteration 1: Assign points to nearest cluster

Step 1: Calculate distances

Point	Distance to C1(3,9)	Distance to C2(10,10)	Distance to C3(2,3)	Assigned Cluster
B1(3,9)	0	$7 + 1 = 8$	$1 + 6 = 7$	C1
B2(4,8)	$1 + 1 = 2$	$6 + 2 = 8$	$2 + 5 = 7$	C1
B3(10,10)	$7 + 1 = 8$	0	$8 + 7 = 15$	C2
B4(9,8)	$6 + 1 = 7$	$1 + 2 = 3$	$7 + 5 = 12$	C2
B5(2,3)	$1 + 6 = 7$	$8 + 7 = 15$	0	C3
B6(3,2)	$0 + 7 = 7$	$7 + 8 = 15$	$1 + 1 = 2$	C3
B7(8,9)	$5 + 0 = 5$	$2 + 1 = 3$	$6 + 6 = 12$	C2
B8(1,4)	$2 + 5 = 7$	$9 + 6 = 15$	$1 + 1 = 2$	C3

Step 2: New cluster members

- ¶ Cluster 1 (C1): B1, B2
- ¶ Cluster 2 (C2): B3, B4, B7
- ¶ Cluster 3 (C3): B5, B6, B8

Step 3: Compute new cluster centers (mean of points)

¶ Cluster 1:

$$x = (3+4)/2 = 3.5, y = (9+8)/2 = 8.5$$

$$\text{New C1} = (3.5, 8.5)$$

¶ Cluster 2:

$$x = (10+9+8)/3 = 27/3 = 9, y = (10+8+9)/3 = 27/3 = 9$$

$$\text{New C2} = (9, 9)$$

¶ **Cluster 3:**

$$x = (2+3+1)/3 = 6/3 = 2, y = (3+2+4)/3 = 9/3 = 3 \quad x = (2 + 3 + 1)/3 = 6/3 = 2, \quad y = (3 + 2 + 4)/3 = 9/3 = 3$$

New C3 = (2, 3)

Iteration 2: Reassign points to nearest cluster

Point	Distance to C1(3.5,8.5)	Distance to C2(9,9)	Distance to C3(2,3)	Assigned Cluster
B1(3,9)	$0.5 + 0.5 = 1$	$6 + 1 = 7$	$1 + 6 = 7$	C1
B2(4,8)	$0.5 + 0.5 = 1$	$5 + 1 = 6$	$2 + 5 = 7$	C1
B3(10,10)	$6.5 + 1.5 = 8$	$1 + 1 = 2$	$8 + 7 = 15$	C2
B4(9,8)	$5.5 + 0.5 = 6$	$0 + 1 = 1$	$7 + 5 = 12$	C2
B5(2,3)	$1.5 + 5.5 = 7$	$7 + 6 = 13$	0	C3
B6(3,2)	$0.5 + 6.5 = 7$	$6 + 7 = 13$	$1 + 1 = 2$	C3
B7(8,9)	$4.5 + 0.5 = 5$	$1 + 0 = 1$	$6 + 6 = 12$	C2
B8(1,4)	$2.5 + 4.5 = 7$	$8 + 5 = 13$	$1 + 1 = 2$	C3

Step 4: Compute final cluster centers

- ¶ **Cluster 1:** B1, B2 → Center = (3.5, 8.5)
- ¶ **Cluster 2:** B3, B4, B7 → Center = (9, 9)
- ¶ **Cluster 3:** B5, B6, B8 → Center = (2, 3)

[Exercise 2: K-Means Clustering \(Two Clusters\)](#)

Use K-Means Algorithm to create two clusters.

Assume P1 (3, 4) and P4 (8, 7) are the initial centers of the two clusters.

Use Manhattan Distance:

$$P(a, b) = |x_2 - x_1| + |y_2 - y_1|$$

Given Points:

P1 (3, 4)

P2 (2, 5)

P3 (4, 3)

P4 (8, 7)

P5 (7, 8)

P6 (9, 6)

P7 (1, 2)

P8 (6, 5)

Required:

1. Assign each point to the nearest cluster center (Iteration 1).
2. Recalculate the new cluster centers.
3. Perform Iteration 2.
4. Write the final cluster centers after the second iteration.

Solution

Point	Coordinates
P1	(3, 4)
P2	(2, 5)
P3	(4, 3)
P4	(8, 7)
P5	(7, 8)
P6	(9, 6)
P7	(1, 2)
P8	(6, 5)

Step 0: Initial Cluster Centers

Let's choose initial centers as:

$$\text{¶ } C1 = P1 (3, 4)$$

$$\text{¶ } C2 = P4 (8, 7)$$

$$\text{¶ } C3 = P7 (1, 2)$$

Iteration 1: Assign Points to Nearest Cluster

We calculate **Manhattan distance**:

$$P(a,b)=|x_2 - x_1| + |y_2 - y_1| \quad P(a,b)=|x_2-x_1|+|y_2-y_1|$$

Point	Distance to C1(3,4)	Distance to C2(8,7)	Distance to C3(1,2)	Assigned Cluster
P1(3,4)	0	8	4	C1
P2(2,5)	2	8	4	C1
P3(4,3)	2	8	4	C1
P4(8,7)	8	0	12	C2
P5(7,8)	8	2	12	C2
P6(9,6)	8	2	12	C2
P7(1,2)	4	12	0	C3
P8(6,5)	4	4	8	C1 (tie: choose closer to C1 by convention)

Step 2: Compute New Cluster Centers

C1: P1, P2, P3, P8

$$x=(3+2+4+6)/4=15/4=3.75 \quad y=(4+5+3+5)/4=17/4=4.25 \quad x=(3+2+4+6)/4=15/4=3.75 \quad y=(4+5+3+5)/4=17/4=4.25$$

New **C1** = (3.75, 4.25)

C2: P4, P5, P6

$$x = (8+7+9)/3 = 24/3 = 8 \quad y = (7+8+6)/3 = 21/3 = 7$$

$$x = (8+7+9)/3 = 24/3 = 8 \quad y = (7+8+6)/3 = 21/3 = 7$$

New C2 = (8, 7)

C3: P7

$$x = 1, y = 2 \quad x = 1, y = 2 \quad x = 1, y = 2$$

New C3 = (1, 2)

Iteration 2: Reassign Points to Nearest Cluster

Point	Distance to C1(3.75,4.25)	Distance to C2(8,7)	Distance to C3(1,2)	Assigned Cluster
P1(3,4)	$0.75 + 0.25 = 1$	$5+3=8$	$2+2=4$	C1
P2(2,5)	$1.75+0.75=2.5$	$6+2=8$	$1+3=4$	C3 (closer now)
P3(4,3)	$0.25+1.25=1.5$	$4+4=8$	$3+1=4$	C1
P4(8,7)	$4.25+2.75=7$	0	$7+5=12$	C2
P5(7,8)	$3.25+3.75=7$	$1+1=2$	$6+6=12$	C2
P6(9,6)	$5.25+1.75=7$	$1+1=2$	$8+4=12$	C2
P7(1,2)	$2.75+2.25=5$	$7+5=12$	0	C3
P8(6,5)	$2.25+0.75=3$	$2+2=4$	$5+3=8$	C1

Step 3: Compute New Centers After Iteration 2

C1: P1, P3, P8

$$x = (3 + 4 + 6) / 3 = 13 / 3 \approx 4.33 \quad y = (4 + 3 + 5) / 3 = 12 / 3 = 4$$

$$x = (3+4+6)/3 = 13/3 \approx 4.33 \quad y = (4+3+5)/3 = 12/3 = 4$$

New C1 = (4.33, 4)

C2: P4, P5, P6

$$x=8, y=7 \quad x = 8, y = 7 \quad x=8, y=7$$

New **C2** = (8,7)

C3: P2, P7

$$x=(2+1)/2=1.5, y=(5+2)/2=3.5 \quad x = (2+1)/2=1.5, y=(5+2)/2=3.5$$

$$x=(2+1)/2=1.5, y=(5+2)/2=3.5$$

New **C3** = (1.5,3.5)

Final Cluster Centers After Second Iteration:

$$\text{¶ } \mathbf{C1} = (4.33, 4)$$

$$\text{¶ } \mathbf{C2} = (8, 7)$$

$$\text{¶ } \mathbf{C3} = (1.5, 3.5)$$

Exercise 3:

Given a dataset of 15 persons with their Jobs and Pets, the k-modes algorithm can be used to group similar objects based on categorical features. The k-modes algorithm is very similar to k-means, but instead of using numerical distances and means, it uses a simple distance measure and the mode of the cluster. Formally, for two objects X and Y described by m categorical features, the distance between them is calculated by counting the number of features in which X and Y differ. This trivial distance measure allows us to cluster objects like Jobs and Pets, where averaging or numerical operations are not meaningful. The algorithm starts with k initial modes (cluster centers), assigns each object to the cluster with the nearest mode, updates the cluster modes by selecting the most frequent values of each feature, and repeats these steps until the clusters no longer change.

Person	Job	Department
P1	Teacher	Education
P2	Engineer	Technology
P3	Doctor	Healthcare
P4	Artist	Creative
P5	Lawyer	Legal
P6	Nurse	Healthcare
P7	Chef	Hospitality
P8	Pilot	Aviation
P9	Musician	Creative
P10	Scientist	Technology
P11	Photographer	Creative
P12	Writer	Creative
P13	Farmer	Agriculture
P14	Designer	Creative
P15	Architect	Architecture

K-Modes Distance Measure

For categorical data, the **distance between two objects X and Y** is calculated as the **number of mismatches in their attributes**.

2. Step 1: Select Initial Modes ($k = 3$)

Let's pick **3 initial cluster modes** (randomly selected):

- ¶ **Mode 1:** P1 (Teacher, Education)
- ¶ **Mode 2:** P2 (Engineer, Technology)
- ¶ **Mode 3:** P4 (Artist, Creative)

3. Step 2: Iteration 1 – Assign Persons to Clusters

Distance calculation:

- ¶ Distance = **number of mismatched features** (Job and Department)
- ¶ Example: P3 (Doctor, Healthcare) distances:
 - C1 (Teacher, Education): 2 mismatches
 - C2 (Engineer, Technology): 2 mismatches
 - C3 (Artist, Creative): 2 mismatches
- ¶ Assign **P3** arbitrarily to **C1**.

Cluster Mode	Distance (# mismatches)
Mode 1 (Teacher, Education)	2
Mode 2 (Engineer, Technology)	2
Mode 3 (Artist, Creative)	2

- ¶ Since all distances are equal, assign arbitrarily to **Mode 1** for balance.

Repeat for all 15 persons. A possible **Iteration 1 assignment**:

Cluster	Persons
C1	P1, P3, P5, P6, P13, P15
C2	P2, P10, P7, P8
C3	P4, P9, P11, P12, P14

4. Step 3: Iteration 1 – Update Cluster Modes

Compute **mode (most frequent Job and Department)** for each cluster:

- ¶ **Cluster 1 Mode (C1):**
 - Jobs: Teacher, Doctor, Lawyer, Nurse, Farmer, Architect → Mode = **Doctor**
 - Department: Education, Healthcare, Legal, Healthcare, Agriculture, Architecture → Mode = **Healthcare**
- ¶ **Cluster 2 Mode (C2):**
 - Jobs: Engineer, Scientist, Chef, Pilot → Mode = **Engineer**

- Department: Technology, Technology, Hospitality, Aviation → Mode = **Technology**

¶ **Cluster 3 Mode (C3):**

- Jobs: Artist, Musician, Photographer, Writer, Designer → Mode = **Artist**
- Department: All Creative → Mode = **Creative**

5. Step 4: Iteration 2 – Reassign Persons

Compute distance to **new cluster modes** and assign to the nearest cluster:

¶ **Distance = number of mismatched attributes**

¶ Example: P12 (Writer, Creative) distances:

- C1 (Doctor, Healthcare) = 2 mismatches → assign to C3
- C2 (Engineer, Technology) = 2 mismatches
- C3 (Artist, Creative) = 1 mismatch → **assign to C3**

After reassignment, **Iteration 2 clusters**:

Cluster	Persons	Mode
C1	P1, P3, P5, P6, P13, P15	Job: Doctor / Department: Healthcare
C2	P2, P10, P7, P8	Job: Engineer / Department: Technology
C3	P4, P9, P11, P12, P14	Job: Artist / Department: Creative

6. Step 5: Final Modes (After Iteration 2)

- ¶ **Cluster 1 Mode:** Doctor, Healthcare
- ¶ **Cluster 2 Mode:** Engineer, Technology
- ¶ **Cluster 3 Mode:** Artist, Creative

Cluster	Mode (Job, Department)	Members
C1	Doctor, Healthcare	P1, P3, P5, P6, P13, P15
C2	Engineer, Technology	P2, P10, P7, P8
C3	Artist, Creative	P4, P9, P11, P12, P14

Questions

1. What is clustering in unsupervised learning?
2. How is clustering different from classification?
3. Why are cluster labels sometimes called "unknown" in unsupervised learning?
4. What makes a good cluster?
5. What is the difference between **partitional (flat) clusters** and **hierarchical clusters**?
6. Explain **non-exclusive clusters** and **fuzzy clustering**.
7. How can hierarchical clusters be represented visually?
8. Give an example of a real-world scenario where hierarchical clustering would be more suitable than partitional clustering.
9. Name three distance metrics commonly used in clustering.
10. How does the choice of distance metric affect the clustering results?
11. Why might standardizing features be important before clustering?
12. Describe the basic steps of the k-means clustering algorithm.
13. What is the **SSE (sum of squared errors)**, and how is it used in k-means?
14. Why is k-means sensitive to initial cluster prototypes?
15. What is the purpose of running k-means multiple times with different starting points?
17. What is the difference between **agglomerative** and **divisive** hierarchical clustering?
18. How is the dendrogram interpreted in hierarchical clustering?
19. What is a potential drawback of hierarchical clustering for large datasets?
20. Give two examples of fields where clustering is used and explain the purpose.

Chapter 6: Practice Questions

Question1:

Title: Predicting if a Person Will Go

You are given data about people with the following attributes:

- ¶ **Age:** Age in years
- ¶ **Experience:** Years of experience
- ¶ **Rank:** Job rank (1 to 5)
- ¶ **Nationality:** UK, USA, N
- ¶ **Go:** Whether the person will go (YES/NO)

Tasks:

1. Load the dataset (`data.csv`) into Python using pandas.
2. Encode categorical features (`Nationality`) and the target (`Go`) into numbers.
 - UK = 0, USA = 1, N = 2
 - YES = 1, NO = 0
3. Use the features `Age`, `Experience`, `Rank`, `Nationality` to train a **Decision Tree Classifier** to predict `Go`.
4. Train the model on the full dataset.
5. Visualize the decision tree with colors showing the majority class in each node.
6. Use the trained model to predict whether a **new person** will go:
 - Age = 27, Experience = 3, Rank = 4, Nationality = USA

Mini Dataset (`data.csv`)

You can create a CSV file with this data:

```
Age,Experience,Rank,Nationality,Go
25,2,3,USA,YES
30,5,4,UK,NO
22,1,2,N,YES
28,3,4,USA,YES
35,10,5,UK,NO
40,12,5,UK,NO
21,1,1,N,YES
29,4,3,USA,YES
33,7,4,UK,NO
26,2,2,N,YES
```

How to Save

1. Open Excel or Google Sheets.
2. Copy the table above including headers.
3. Save it as **data.csv** in the same folder as your Python script.

Python Code

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# 1. Create the dataset directly
data = {
    'Age': [25, 30, 22, 35, 28, 40, 23, 32, 26, 29],
    'Experience': [2, 5, 1, 10, 4, 15, 2, 6, 3, 5],
    'Rank': [3, 4, 2, 5, 3, 6, 2, 4, 3, 3],
    'Nationality': ['UK', 'USA', 'N', 'UK', 'USA', 'UK',
    'N', 'USA', 'UK', 'N'],
    'Go': ['YES', 'NO', 'YES', 'NO', 'YES', 'NO', 'YES',
    'NO', 'YES', 'NO']
}

df = pd.DataFrame(data)

# 2. Encode categorical columns
df['Nationality'] = df['Nationality'].map({'UK':0,
    'USA':1, 'N':2})
df['Go'] = df['Go'].map({'YES':1, 'NO':0})

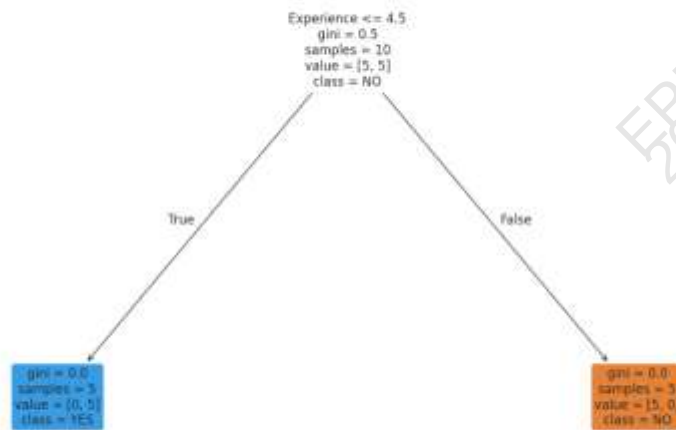
# 3. Define features and target
features = ['Age', 'Experience', 'Rank', 'Nationality']
X = df[features]
y = df['Go']

# 4. Train Decision Tree
dtree = DecisionTreeClassifier(random_state=0)
dtree.fit(X, y)
```

```
# 5. Visualize the tree
plt.figure(figsize=(20,12))
plot_tree(dtree, feature_names=features,
          class_names=['NO', 'YES'],
          filled=True, rounded=True, fontsize=12)
plt.show()

# 6. Predict for a new person
new_person = [[27, 3, 4, 1]] # Age, Experience, Rank,
Nationality(USA=1)
prediction = dtree.predict(new_person)

print("Prediction (0=NO, 1=YES):", prediction[0])
```



Question 2: Predicting Loan Approval

You are given a dataset of loan applicants with the following columns:

- ¶ **Age:** Age of the applicant in years
- ¶ **Income:** Annual income in \$1000s
- ¶ **CreditScore:** Credit score (300 to 850)
- ¶ **Married:** Whether the applicant is married (Yes/No)
- ¶ **Approved:** Whether the loan was approved (Yes/No)

Tasks:

1. Load the dataset (`loan_data.csv`) into Python using pandas.
2. Encode categorical features (Married) and target (Approved) into numbers:
 - Yes = 1, No = 0
3. Use Age, Income, CreditScore, Married as **features** and Approved as the **target**.
4. Train a **Decision Tree Classifier** on the dataset.
5. Visualize the trained tree, with nodes colored by the majority class.
6. Use the model to predict whether a **new applicant** will get approval:
 - Age = 30, Income = 50, CreditScore = 720, Married = Yes

Mini Dataset (`loan_data.csv`)

```
Age,Income,CreditScore,Married,Approved
25,40,650,Yes,Yes
45,80,700,No,Yes
30,50,600,Yes,No
35,60,720,Yes,Yes
28,35,580,No,No
50,90,710,Yes,Yes
40,75,690,No,Yes
22,30,550,No,No
33,55,640,Yes,Yes
38,65,680,Yes,Yes
```

Python Code

```
import pandas as pd

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score,
confusion_matrix

import matplotlib.pyplot as plt
```

```
# 1 Create Dataset
```

```
data = {  
    'Age':  
    [25,30,22,35,28,40,23,45,32,27,29,38,24,41,26,33,36,21,42,  
    31],  
    'Income':  
    [40,60,25,80,55,90,30,100,65,45,50,85,35,95,48,70,88,20,92,  
    58],  
    'CreditScore':  
    [680,750,600,820,700,850,620,900,720,690,710,780,640,370,7  
    05,730,800,580,860,715],  
    'Married':  
    ['No','Yes','No','Yes','Yes','Yes','No','Yes','Yes','No',  
  
    'Yes','Yes','No','Yes','No','Yes','Yes','No','Yes','Yes'],  
    'Approved':  
    ['No','Yes','No','Yes','Yes','Yes','No','Yes','Yes','No',  
  
    'Yes','Yes','No','Yes','No','Yes','Yes','No','Yes','Yes']  
}
```

```
df = pd.DataFrame(data)
```

```
# 2 Encode categorical columns
```

```
df['Married'] = df['Married'].map({'Yes':1, 'No':0})  
df['Approved'] = df['Approved'].map({'Yes':1, 'No':0})
```

```

# 3 Define features and target
features = ['Age', 'Income', 'CreditScore', 'Married']
X = df[features]
y = df['Approved']

# 4 Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

# 5 Train Decision Tree
dtree = DecisionTreeClassifier(random_state=0)
dtree.fit(X_train, y_train)

# 6 Evaluate Model
y_pred = dtree.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test,
y_pred))

# 7 Visualize the tree
plt.figure(figsize=(20,12))

```

```

plot_tree(dtrees, feature_names=features,
          class_names=['No', 'Yes'],
          filled=True, rounded=True, fontsize=10)

plt.show()

```

```

# 8 Predict for new applicant

new_applicant = [[30, 50, 720, 1]] # Age, Income,
CreditScore, Married(Yes=1)

prediction = dtrees.predict(new_applicant)

print("Prediction (0=No, 1=Yes):", prediction[0])

```

OUTPUT



Question 3:

- ¶ Creates a small dataset of 10 employees.
- ¶ Encodes categorical data (Nationality and Go).
- ¶ Trains a `DecisionTreeClassifier`.
- ¶ Plots the decision tree with labeled features.
- ¶ Predicts whether a **new employee will attend** based on their features.

Python Code:

```
# Import libraries

import pandas as pd

from sklearn.tree import DecisionTreeClassifier

from sklearn import tree

import matplotlib.pyplot as plt


# Create a larger dataset

data = {

    'Age': [25,30,22,35,28,40,23,32,26,29,

            31,27,24,36,38,21,33,34,29,30,

            26,28,37,39,22,41,27,35,24,32],

    'Experience': [2,5,1,10,4,15,2,6,3,5,

                  7,3,2,11,12,1,8,9,4,6,

                  3,4,13,14,2,16,5,10,2,7],
```

```
'Rank': [3,4,2,5,3,6,2,4,3,3,  
         4,3,2,5,6,1,4,5,3,4,  
         3,3,6,6,2,7,4,5,2,4],
```

```
'Nationality': ['UK','USA','N','UK','USA','UK','N','USA','UK','N',  
               'USA','UK','N','USA','UK','N','USA','UK','N','USA',  
               'UK','USA','UK','USA','N','UK','USA','UK','N','USA'],
```

```
'Go': ['YES','NO','YES','NO','YES','NO','YES','NO','YES','NO',  
       'NO','YES','YES','NO','NO','YES','NO','NO','YES','NO',  
       'YES','YES','NO','NO','YES','NO','YES','NO','YES','NO']
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# Encode categorical variables
```

```
df['Nationality'] = df['Nationality'].map({'UK': 0, 'USA': 1, 'N': 2})
```

```
df['Go'] = df['Go'].map({'YES': 1, 'NO': 0})
```

```
# Features and target
```

```
features = ['Age', 'Experience', 'Rank', 'Nationality']
```

```
X = df[features]
```

```
y = df['Go']
```

```
# Create and train Decision Tree
```

```
dtree = DecisionTreeClassifier(max_depth=4, random_state=0)

dtree.fit(X, y)

# Plot the tree

plt.figure(figsize=(15,10))

tree.plot_tree(

    dtree,

    feature_names=features,

    class_names=['NO','YES'],

    filled=True

)

plt.show()

# Predict for new employee

new_employee = [[28, 4, 3, 1]] # USA = 1

prediction = dtree.predict(new_employee)

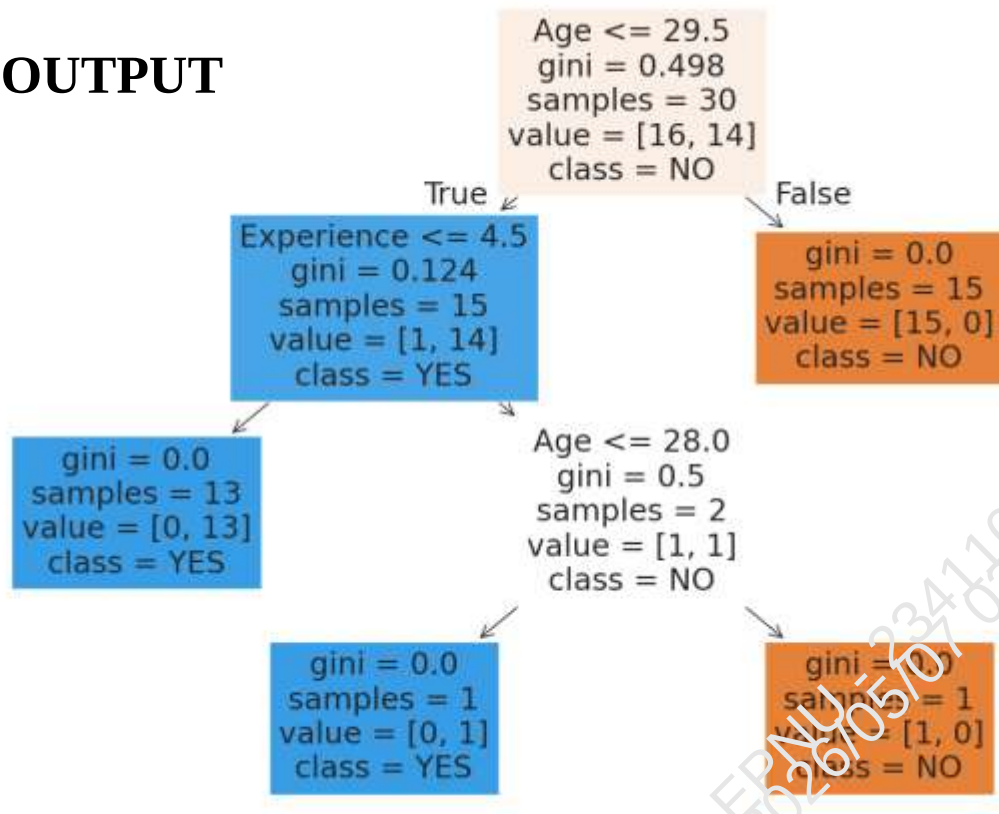
if prediction[0] == 1:

    print("Prediction: YES (will attend)")

else:

    print("Prediction: NO (will not attend)")
```

OUTPUT



Question 4: Customer Segmentation Using K-Means Clustering

A retail store wants to **segment its customers** based on their **Annual Income** and **Spending Score** to plan marketing campaigns.

You are given the following mini dataset:

CustomerID	Age	Annual_Income	Spending_Score
1	19	15	39
2	21	15	81
3	20	16	6
4	23	16	77
5	31	17	40
6	22	17	76
7	35	18	6
8	23	18	94
9	64	19	3
10	30	19	72

Tasks

1. Use **K-Means clustering** to segment customers into **3 clusters**.
2. Plot the clusters in a 2D plot (**Annual_Income vs Spending_Score**).
3. Identify which cluster corresponds to **high-spending customers**.

Python Code

```
# Import libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

# Create dataset

data = {

    'CustomerID': range(1, 11),

    'Age': [19, 21, 20, 23, 31, 22, 35, 23, 64, 30],

    'Annual_Income': [15, 15, 16, 16, 17, 17, 18, 18, 19, 19],

    'Spending_Score': [39, 81, 6, 77, 40, 76, 6, 94, 3, 72]

}

df = pd.DataFrame(data)

# Select features for clustering

X = df[['Annual_Income', 'Spending_Score']]

# Create KMeans object and fit

kmeans = KMeans(n_clusters=3, random_state=0)
```

```

df['Cluster'] = kmeans.fit_predict(X)

# Plot the clusters

plt.figure(figsize=(8,6))

colors = ['red', 'green', 'blue']

for i in range(3):

    cluster = df[df['Cluster'] == i]

    plt.scatter(cluster['Annual_Income'], cluster['Spending_Score'],

                s=100, c=colors[i], label=f'Cluster {i}')

# Plot centroids

plt.scatter(kmeans.cluster_centers_[ :,0], kmeans.cluster_centers_[ :,1],

            s=300, c='yellow', label='Centroids', marker='*')

plt.xlabel('Annual Income (k$)')

plt.ylabel('Spending Score')

plt.title('Customer Segments (K-Means)')

plt.legend()

plt.show()

# Print cluster assignments

print(df[['CustomerID', 'Annual_Income', 'Spending_Score', 'Cluster']])

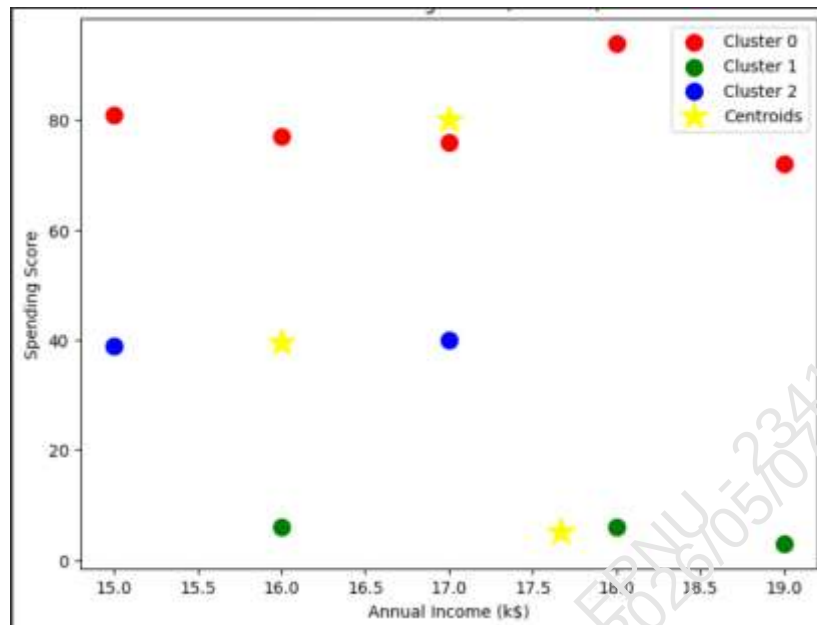
```

Explanation

1. We cluster customers using **Annual Income** and **Spending Score**.
2. `KMeans(n_clusters=3)` divides them into 3 clusters.
3. The **scatter plot** shows the clusters with centroids in yellow.

4. You can identify **high-spending customers** by looking at the cluster with high Spending_Score.

Output:



Question5: Movie Lovers Segmentation Using K-Means

A streaming service wants to **segment its users** based on their **hours watched per week** and **number of movies rated** to improve recommendations and marketing strategies.

You are given the following mini dataset:

UserID	Hours_Watched	Movies_Rated
1	5	2
2	10	5
3	2	1
4	8	4
5	20	15
6	18	12
7	25	20
8	3	2
9	12	6

UserID	Hours_Watched	Movies_Rated
10	22	18
11	7	3
12	30	25
13	1	1
14	15	10
15	28	22

Tasks

1. Use **K-Means clustering** to segment users into **3 clusters**.
2. Plot the clusters in a 2D plot (**Hours_Watched vs Movies_Rated**).
3. Identify which cluster corresponds to **heavy viewers**.

Import libraries

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

Create the dataset

```
data = {
    'UserID': range(1, 16),
    'Hours_Watched': [5, 10, 2, 8, 20, 18, 25, 3, 12, 22, 7, 30, 1, 15, 28],
    'Movies_Rated': [2, 5, 1, 4, 15, 12, 20, 2, 6, 18, 3, 25, 1, 10, 22]
}
```

df = pd.DataFrame(data)

Select features for clustering

X = df[['Hours_Watched', 'Movies_Rated']]


```

# Apply K-Means clustering

kmeans = KMeans(n_clusters=3, random_state=0)

df['Cluster'] = kmeans.fit_predict(X)


# Plot the clusters

plt.figure(figsize=(8,6))

colors = ['red', 'green', 'blue']

for i in range(3):

    cluster = df[df['Cluster'] == i]

    plt.scatter(cluster['Hours_Watched'], cluster['Movies_Rated'],

                s=100, c=colors[i], label=f'Cluster {i}')


# Plot centroids

plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],

            s=300, c='yellow', label='Centroids', marker='*')

plt.xlabel('Hours Watched per Week')

plt.ylabel('Movies Rated')

plt.title('User Segments (K-Means)')

plt.legend()

plt.show()

```

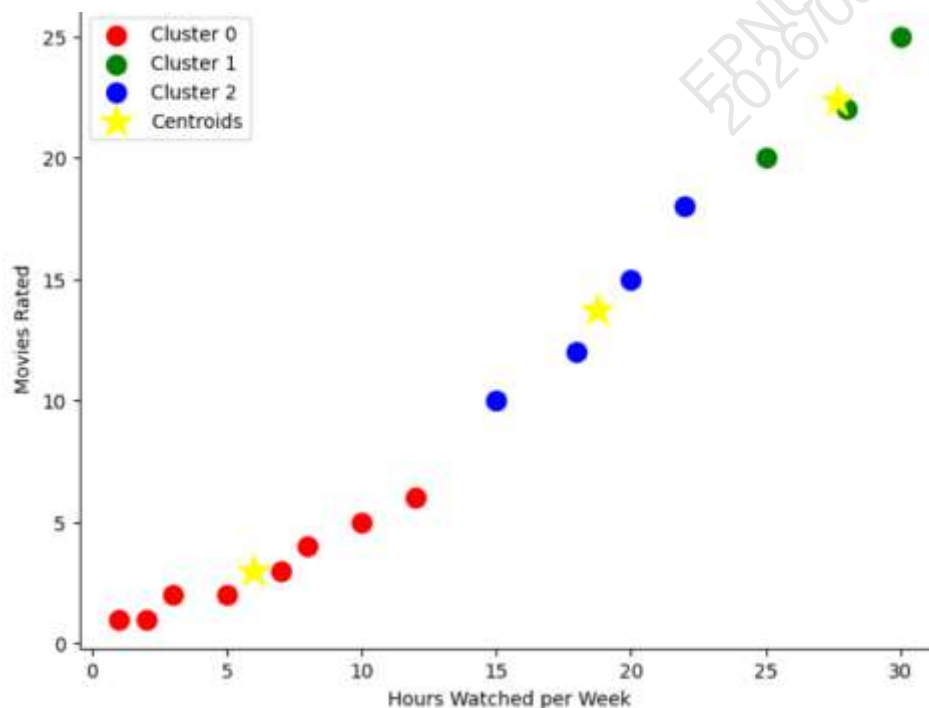
```
# Print cluster assignments
```

```
print(df[['UserID','Hours_Watched','Movies_Rated','Cluster']])
```

Explanation

1. We cluster users based on **Hours_Watched** and **Movies_Rated**.
2. The **scatter plot** shows 3 clusters and their centroids.
3. The cluster with **high Hours_Watched** and **high Movies_Rated** represents **heavy viewers**.
4. The other clusters correspond to **casual viewers** and **light viewers**.

Output:

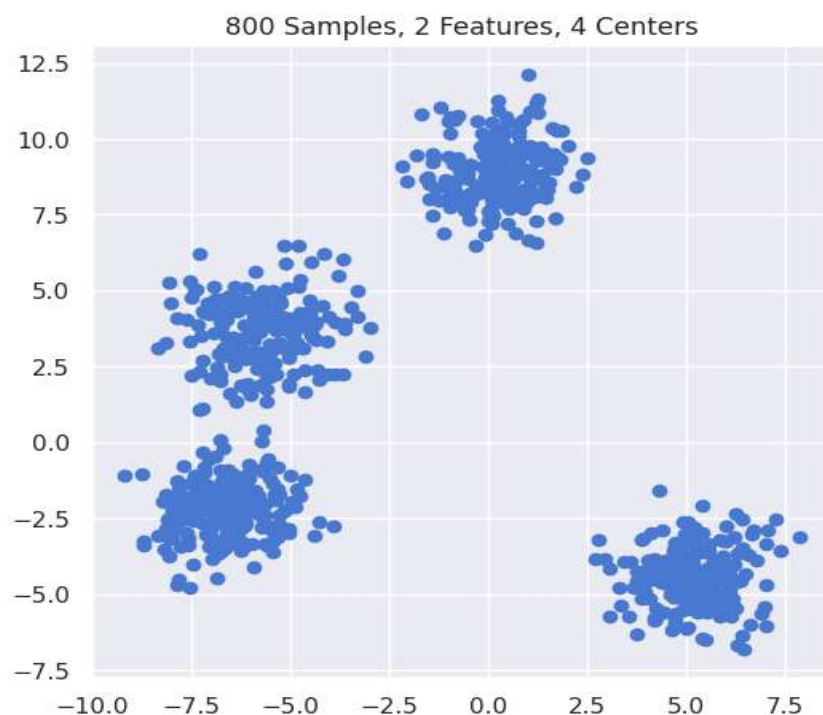


Question5: Problem: K-Means Clustering on Generated Data

```
import numpy as np
```

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
# Change number of samples and features
X, y = make_blobs(n_samples=800, # was 500 → now 800
                  n_features=2, # keep 2 features for 2D plot
                  centers=4,
                  random_state=23)
```

```
plt.figure(figsize=(6,6))
plt.grid(True)
plt.scatter(X[:,0], X[:,1])
plt.title("800 Samples, 2 Features, 4 Centers")
plt.show()
```



Questions:

1-A company wants to predict whether an employee will be **Promoted** or **Not Promoted** based on the following features:

- ¶ Age
- ¶ Years of Experience
- ¶ Performance Score (1 to 10)
- ¶ Overtime (Yes/No)

Train a Decision Tree Classifier. And Visualize the decision tree.

You are given the following dataset:

Age	Experience	Performance	Overtime	Promoted
25	2	6	Yes	No
30	5	8	No	Yes
22	1	5	Yes	No
35	10	9	No	Yes
28	4	7	Yes	Yes
40	15	9	No	Yes
23	2	6	Yes	No
32	6	8	No	Yes
26	3	7	Yes	Yes
29	5	6	No	No

2-A mall wants to divide customers into groups based on their:

- ¶ Annual Income (k\$)
- ¶ Spending Score (1 to 100)

You are given the following dataset:

Customer	Income	Spending Score
1	15	39
2	16	81
3	17	6
4	18	77
5	19	40
6	20	76
7	21	6
8	22	94
9	23	3
10	24	72

Required:

- ¶ Apply K-Means clustering with **K = 2**
- ¶ Find cluster centers
- ¶ Add cluster labels to dataset
- ¶ Visualize the clusters using a scatter plot

3-A university wants to group students based on:

- ¶ Study Hours per week
- ¶ Exam Score

You are given the dataset:

Student	StudyHours	ExamScore
1	2	50
2	3	55
3	4	60
4	5	65
5	6	70
6	7	75
7	8	80

Student	StudyHours	ExamScore
8	9	85
9	10	90
10	1	45

Required:

1. Use the **Elbow Method** to determine the optimal number of clusters.
2. Apply K-Means using the best K.
3. Print cluster centers.
4. Add cluster labels to the dataset.
5. Visualize the clusters in a scatter plot.

EPNU - 23411014
2026/05/07 03:28